

密级：公开

北京邮电大学

硕士学位论文



题目： 基于深度学习的源码漏洞检测鲁棒性研究

学 号： 2020110698

姓 名： 聂旭

学科专业： 计算机科学与技术

培养方式： 全日制

导 师： 王尚广

学 院： 计算机学院（国家示范性软件学院）

2023 年 5 月 30 日

中国·北京

密级：公开

北京邮电大学

硕士学位论文（学术学位）



题目：基于深度学习的源码漏洞检测鲁棒性研究

学 号：2020110698

姓 名：聂旭

学科专业：计算机科学与技术

培养方式：全日制

导 师：王尚广

学 院：计算机学院（国家示范性软件学院）

2023 年 5 月 30 日



BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

Master Dissertation (Academic Degree)

RESEARCH ON ROBUSTNESS OF SOURCE CODE VULNERABILITY DETECTION BASED ON DEEP LEARNING

Student ID: 2020110698

Author: Xu Nie

Subject: Computer Science and Technology

Training Mode: Full-Time

Supervisor: Shangguang Wang

Institute: School of Computer Science (National
Pilot Software Engineering school)

May 30, 2023

答辩委员会名单

职务	姓 名	职 称	工 作 单 位
主席	孙其博	副教授	北京邮电大学
委员	曹素芝	研究员	中国科学院空间应用中心
委员	张宇超	副教授	北京邮电大学
秘书	张乙然	副教授	北京邮电大学
答辩日期	2023 年 5 月 30 日		

基于深度学习的源码漏洞检测鲁棒性研究

摘 要

随着软件供应链的日益复杂,开源软件中存在的源代码安全漏洞也会造成更加深远的恶劣影响。因此,研究一个能够精准检测开源软件中漏洞的工具变得越来越迫切。由于现阶段 AI 技术在 CV 和 NLP 领域的兴起,引发了众多研究者将深度学习技术应用到源代码漏洞检测中去。然而,大家都在朝着提高漏洞检测模型的准确性方向努力,而忽略了模型鲁棒性这个最重要的问题。在软件安全领域中,一个不够鲁棒的模型是无法真正应用的,一旦在生产环境中被攻击者利用,将会造成不可估量的后果。

为了防止出现此类后果,本文参考了大量的漏洞检测相关研究工作,并做出了如下贡献:首先指出了基于深度学习的源码漏洞检测任务中研究者们常用的漏洞数据集中存在噪声且数据集样本模式单一的问题,揭露了导致模型鲁棒性不足的重要原因。其次探究了噪声的来源及噪声对模型训练的实际影响,并提出了基于集成学习和置信剪枝的去噪方法,能够通过去噪来减轻噪声对模型的影响。然后针对漏洞数据集样本单一的问题,提出了基于语义保护代码变换的数据增强方法,通过数据增强技术能够丰富数据集中的样本模式且提升模型的性能。最后,为了满足领域内对于一个合理的漏洞检测模型鲁棒性度量方法的需求,本文提出了一个多维度的漏洞检测模型鲁棒性度量方法,对一些经典的漏洞检测模型进行了多维度的鲁棒性评估,并且通过此评估方法也证明了去噪和数据增强对提升模型鲁棒性有一定效果,为优化漏洞检测模型鲁棒性提供了两条可行的方案。

关键词 漏洞检测 鲁棒性 数据集 去噪 数据增强

RESEARCH ON ROBUSTNESS OF SOURCE CODE VULNERABILITY DETECTION BASED ON DEEP LEARNING

ABSTRACT

As the software supply chain becomes increasingly complex, source code security vulnerabilities in open source software will also have far-reaching adverse effects. Therefore, it is becoming more and more urgent to invent a tool that can accurately detect vulnerabilities in open source software. Due to the rise of AI technology in the CV and NLP fields at this stage, many researchers have applied deep learning technology to source code vulnerability detection. However, everyone is working towards improving the accuracy of vulnerability detection models, ignoring the most important issue of model robustness. In the field of software security, a model that is not robust enough cannot be truly applied. Once exploited by attackers in the production environment, it will cause immeasurable consequences.

In order to prevent such consequences, this paper refers to many research works related to vulnerability detection, and makes the following contributions: First of all, this work points out the problems of noise and single sample mode in vulnerability datasets commonly used by researchers in the deep-learning-based vulnerability detection task, and reveals the important reasons for the low robustness of the model. Secondly, this topic explores the source of noise and the actual impact of noise on model training, and proposes a denoising method based on ensemble learning and confidence pruning, which can reduce the impact of noise on the model through denoising. Then, aiming at the problem of single sample pattern in the vulnerability dataset, this topic proposes a data augmentation method for semantic protection code transformation. Through data augmentation, the sample pattern in the dataset can be enriched and the performance of the model can be improved. Through data augmentation,

the problem of low robustness caused by single sample pattern and insufficient data volume is solved. Finally, in order to meet the needs of a reasonable vulnerability detection model robustness measurement method in the field, we propose a multi-dimensional vulnerability detection model robustness measurement method, and conduct a multi-dimensional analysis of some classic vulnerability detection models. With this evaluation method, we also demonstrate that denoising and data augmentation have a significant effect on improving model robustness. And we provide two feasible solutions for optimizing the robustness of the vulnerability detection model.

KEY WORDS vulnerability-detection robustness dataset denoising data-augmentation

目 录

第一章 绪论.....	1
1.1 研究背景.....	1
1.2 研究内容.....	4
1.3 组织结构.....	7
第二章 相关研究及技术	9
2.1 源代码漏洞检测.....	9
2.1.1 静态分析技术.....	9
2.1.2 动态分析技术.....	11
2.1.3 基于深度学习的漏洞检测技术.....	11
2.2 抗噪声学习.....	14
2.3 对抗攻击及防护.....	16
2.3.1 攻击手段.....	16
2.3.2 防御手段.....	16
2.4 本章小结.....	17
第三章 基于集成学习与置信剪枝的漏洞数据集去噪	18
3.1 问题描述.....	18
3.1.1 噪声的普遍性.....	18
3.1.2 漏洞数据集中引入噪声标签的原因.....	20
3.2 去噪算法原理.....	23
3.2.1 置信学习.....	23
3.2.2 差异训练.....	24
3.2.3 基于集成学习和置信剪枝的漏洞数据集去噪	24
3.3 实验设置.....	25
3.3.1 数据集选择.....	25
3.3.2 漏洞检测 BaseLine.....	26
3.3.3 数据标注.....	26
3.3.4 生成噪声标签.....	26
3.3.5 硬件标准.....	28
3.3.6 CL 和 DT 的实现	28
3.3.7 评估指标.....	28
3.4 实验及结论.....	28
3.4.1 噪声对漏洞检测器的影响.....	28
3.4.2 去噪方法的有效性.....	30
3.4.3 漏洞检测模型对去噪方法有效性的影响	31

3.4.4 通过去噪增强漏洞检测模型.....	33
3.5 本章小结.....	34
第四章 基于语义保护代码变换的数据增强	35
4.1 问题描述.....	35
4.2 代码变换算法原理.....	36
4.2.1 代码变换规则.....	36
4.2.2 代码变换策略.....	39
4.2.3 代码变换实现细节.....	40
4.2.4 代码变换优化方案.....	42
4.3 实验设置.....	44
4.4 实验及结论.....	44
4.4.1 数据增强的有效性.....	44
4.4.2 不同程度数据增强的影响.....	46
4.4.3 有针对性的语义保护代码变换面对随机代码变换是否有优势	47
4.5 本章小结.....	48
第五章 多维度的漏洞检测模型鲁棒性度量方法	49
5.1 问题描述.....	49
5.2 多维度度量指标.....	50
5.3 实验设置.....	53
5.3.1 数据集选择.....	53
5.3.2 数据标注.....	54
5.3.3 生成噪声标签.....	54
5.4 实验结果及结论.....	54
5.4.1 不同漏洞检测模型多维度鲁棒性评估	55
5.4.2 去噪对模型鲁棒性的影响.....	56
5.4.3 数据增强对模型鲁棒性的影响.....	57
5.4.4 数据增强结合去噪对模型鲁棒性的影响	58
5.5 本章小结.....	59
第六章 总结与展望	60
6.1 论文总结.....	60
6.2 未来工作展望.....	61
参考文献.....	62

第一章 绪论

1.1 研究背景

随着开源软件的广泛重用，以及软件供应链的日益复杂，开源软件中存在的漏洞也会造成更恶劣的影响，正如通用漏洞披露（CVE）中报告的大量漏洞所证明的那样，检测开源软件中的漏洞变得越来越迫切。众所周知，传统漏洞检测技术的假阳性和假阴性率很高^[1,2,3,4,5]。例如，基于静态分析的漏洞检测工具通常会存在高假阳性（误报）情况，即将非漏洞的源代码误判为漏洞，而基于动态分析的漏洞检测工具则常常存在高假阴性（漏报）情况，即无法检测到许多真正的漏洞。即使经过了长时间的努力，这些工具仍然不够可靠，为开发人员留下了大量的手工开销。由于传统漏洞检测技术不能满足工业界漏洞检测的需求，基于深度学习(DL)的漏洞检测方法开始浮现。特别是在计算机视觉和自然语言处理等领域，深度学习已经取得了巨大的成就，引发了人们对使用深度学习自动高精度检测安全漏洞的兴趣。根据谷歌学者的统计，在 2019 年至 2020 年期间，安全和软件工程领域出现了 92 篇流行论文，这些论文使用深度学习技术来检测不同类型的漏洞。事实上，最近的几项研究^[6,7,8,9,10,11,12,13,14,15,16]已经展现了非常有希望的结果，在漏洞检测方面达到了很高的准确性（高达 95%）。这些方法分别在他们的工作中展现出了不同方面的成功。然而，Chakraborty 等人在工作^[9]中对这些工作在多个漏洞数据集上的有效性进行了深度评估，并发现了一些值得令人深思的问题。首先，他们得出的结论是现有的模型在现实环境中都表现不佳。如果直接使用预训练的模型来检测真实世界的漏洞，漏洞检测模型的性能平均会下降 73%。即使使用真实数据重新训练这些模型，它们的性能也会比报告的结果下降 54%。例如，VulDeePecker^[10]在他们的论文中报道了 86.9% 的精度。然而，在真实世界的数据集上使用 VulDeePecker 预训练的模型时，其检测精度降低到原来的 11.12%，再重新训练后，检测精度提高了 17.68%。其次，Chakraborty 等人给出了可能导致漏洞检测模型不够鲁棒的一些原因。

- 1、**模型的不足**。最流行的模型是基于 token 的，它将代码视为一系列 token，并且没有考虑在漏洞预测中发挥重要作用的语义依赖关系。即使使用了基于图形的模型，它也不会着重于增加漏洞类和非漏洞类之间的分类边界。因此，这些方法的精度和召回率在真实世界的数据场景中都很低。

- 2、**模型学习了不相关的特征**。在研究现有技术所提取的特征(使用最先进的解释技术^[17,18])时,我们发现最先进的模型基本上提取的是与漏洞无关的特征,可能是由于训练数据集中的噪声或其他因素导致的不相关特征。
- 3、**数据重复**。部分现有方法中的训练和测试数据存在重叠(高达 68%);因此,数据重复会导致数据泄露,从而产生了结果偏高的现象。
- 4、**数据不平衡**。由于在现实世界中非漏洞代码的数量远远大于漏洞代码的数量,导致类别不平衡问题无法得到有效缓解。

当把训练好的模型应用到另一个非同源的数据集上时,模型的精度会大幅度的下降。这种不稳定的漏洞检测表现足够表明一点,现有的基于深度学习的漏洞检测模型不够鲁棒。

另外,由于深度学习是数据驱动的,训练一个能够有效检测漏洞的模型需要大量高质量的漏洞数据集。但是,由于漏洞检测的危险性,公开的软件漏洞源码非常有限,因为一旦公开源码细节,漏洞可能被黑客利用获得非法收益。因此,在开放科学领域,真实世界漏洞数据集很难获取,获得高质量标签的漏洞数据集需要大量的知识储备和人力、时间资源的付出。因此,至今还没有一个公开的真实世界漏洞数据集,可以用来评估这些基于深度学习的漏洞检测方法的优劣。目前,大多数方法都是在与真实世界漏洞数据模式相差很大的软件保障参考数据集(SARD)上进行评估,一些方法会在自己的工作中使用自己收集和组织的真实世界数据集进行对比。因此,我们参考了一些在漏洞检测领域表现较好的工作[6,7,8,9,10,11,12,13,14,15,16],并对这些工作使用的数据集和样本标注方法进行了汇总。这些工作包括函数级漏洞检测方法和语句级漏洞检测方法。在这些方法使用的数据集中,SARD 是最受欢迎的数据集,因为 SARD 提供了大量可以研究和学习的人工合成安全缺陷和漏洞。与其他数据集相比,SARD 具有更高质量的标签,因为它是根据专家预定义的漏洞模式自动生成的,并且提供了标注到行的漏洞源代码。但是,SARD 对于训练模型学习到真实世界漏洞特征的贡献非常有限,因为 SARD 的漏洞模式相对简单,与真实世界数据中的漏洞模式差别很大。我们还在 SARD 提供的 XML 文件中发现了一些错误标注,这可能会导致 SARD 产生噪声标签,影响下游模型的训练效果。

除此之外,一些工作^[10,11,12]通常使用国家漏洞数据库(NVD)来获取真实世界的漏洞数据集。NVD 中的每个漏洞都有一个唯一的通用漏洞公开标识符(CVE ID)和通用漏洞类型标识符(CWE ID),用于指示存在问题的漏洞类型。每个漏洞都有多个指向漏洞所在的软件存储库的引用链接。因此,这些工作通过在 NVD 中爬取 CVE,并将 CVE 映射到与漏洞相关的软件更新版本,来获得包含漏洞修复的源代码。然而,NVD 中提供的参考链接并不总是准确的。即使从引用链接

获取的软件更新版本与漏洞修复完全相关,如何在下一步中标记包含漏洞触发的函数/行也是一个需要考虑的问题。这是因为软件更新版本很可能包含与漏洞修复无关的修改。虽然从 NVD 获取的漏洞数据更接近真实数据,但标签质量是不可靠的。

除了以上的常用漏洞数据,Russell^[13]使用了三个开源静态分析器,Clang static Analyzer, Cppcheck 和 Flawfinder 来生成拥有健壮标签的漏洞数据集。然而,众所周知,静态分析工具会产生大量的假阳性结果,这不可避免地会影响其标签质量,在收集结果中引入噪声标签。经过我们对近年来前沿的基于深度学习的漏洞检测方法所使用的数据集的分析,我们发现,这些数据集中都存在着噪声标签,噪声标签会影响模型对真正漏洞特征的学习,最终导致训练完成的模型鲁棒性不足。

最后,虽然在源代码处理领域中,深度学习模型在准确性或 F1-score 方面取得了具有竞争力的表现,但一个健壮的模型应该在输入代码片段受到轻微扰动时做出一致的预测,鲁棒性是衡量源代码漏洞检测模型鲁棒程度的指标。换句话说,当输入的源代码样本在程序语义不变的情况下发生形式上的改变,漏洞检测模型的检测结果应该与源代码发生变换之前保持一致。理想情况下,在程序语义级进行分析是可取的,因为大多数源代码处理任务倾向于将近似代码段的语义含义相关联,例如,代码功能分类和代码克隆检测。

然而,现有的深度学习技术大多工作在具有高度复杂和多样性的代码片段上,这使得模型的鲁棒性更加重要。例如,在漏洞检测系统中,这种自然发生或故意的代码变换可能会欺骗并绕过 DL 漏洞检测系统,将精心设计过的包含漏洞源代码发送到下游处理模块,会引发不可弥补的灾难。这种非鲁棒性对整个系统来说可能是致命的。在实践中,源代码语料库在软件开发过程中自然表现出多样性。例如,具有不同编码风格的开发人员通常以不同的方式实现相同的任务。即使是同一个开发人员,他/她也可以在不同的情况下编写不同的任务(例如,在一周中间的时间或在周末前的星期五)。类似地,相同的算法或功能可能基于不同的库实现,从而导致代码片段的许多变体。为了适应源代码的多样性,理想情况下,漏洞检测模型应该对具有相同语义的代码变体表现一致(即健壮)。否则,如果一个变体即使有一些小的扰动(例如,改变了一些标识符)也会导致不一致的预测,那么漏洞检测模型就不应该被部署,这可能会阻碍它在现实世界软件开发场景中的广泛采用。具体来说,从安全的角度来看,对抗性攻击会有意地对代码进行转换,以创建上述变体(也称为对抗性样本),从而绕过漏洞检测模型,甚至可能导致整个系统的故障。Li 等人^[19]对基于深度学习的漏洞检测模型应对对抗攻击时的鲁棒性进行了研究。首先,为了理解这些检测器的健壮性,他们利用保留语义

的代码转换技术(称为“对抗攻击”)对四个典型的基于 DL 的漏洞检测模型进行对抗攻击。这些漏洞检测模型足够具有代表性,因为它们在不同的检测粒度下运行,使用不同的向量表示方法,并使用了不同的神经网络结构。最终,他们发现一些针对最先进的基于 DL 的漏洞检测模型^[11]的攻击会导致其假阳性率从 7.0% 增加到 19.9%,假阴性率从 9.9% 增加到 68.1%。这说明,现阶段基于深度学习的漏洞检测模型应对对抗攻击时还不够鲁棒。

1.2 研究内容

研究影响模型鲁棒性的原因,并提出解决方案,能够对未来基于深度学习的源码漏洞检测工作给出关键的参考意义,从而提出对开源软件源码中的漏洞真正鲁棒的模型,进一步为解决开源软件中的漏洞检测问题做出贡献。

基于深度学习的模型鲁棒性研究,已经在其他领域备受关注,目前的人工智能领域中针对模型进行改进的鲁棒性分析方法主要分为精确方法和近似方法两大类:精确方法由于其只适用于及小规模的神经网络,且基于深度学习的源码漏洞检测模型通常为大规模的神经网络,因此并不适用于漏洞检测模型中。近似方法中 Lecuyer 等人^[20]提出了 PixelDP,利用差分隐私和模型鲁棒性之间的联系首次提出了“随机平滑”这一概念,提供了对 top-1 预测进行随机平滑处理的鲁棒性保证。具体地说,以测试阶段为例,对于给定的样本 x ,传统的神经网络分类模型将单次预测得到的 argmax 类别作为最终预测结果给出,而随机平滑方法则是利用蒙特卡洛(Monte Carlo)估计方法,通过在样本 x 周围采样 n 个样本(相当于给 x 添加随机噪声)得到它们 argmax 类别的期望值并将其作为最终预测结果。Salman 等人^[21]针对预训练模型提出了一种黑盒随机平滑的方法,通过在现有的图像分类模型之前添加降噪器并使用随机平滑得到新分类模型,在不修改预训练分类模型的情况下保证了对对抗样本具有 ℓ_p 鲁棒性,可用于保护 Azure、Google、AWS 和 ClarifAI 等图像分类 API。Jia 等人^[22]考虑了在实际中应用更为广泛的 top-k 预测鲁棒性。Wang 等人^[23]和 Maurice 等人^[24]基于随机平滑的思想研究了模型对后门攻击(backdoor attack)的鲁棒性。

由于图像的输入样本空间是连续的,与源代码漏洞检测领域中离散的文本输入空间是有极大的差异的,所以,除了针对模型做修改来提高深度学习模型鲁棒性的方法,还有一些针对数据集做处理来提高模型鲁棒性的方法,这类方法更适用于漏洞检测任务,例如 Chakraborty 等人^[25],通过删除数据集中有偏差的样本,并对数据集内部的数据分布进行再平衡,使其基于分类和敏感属性分布平衡,从而解决深度学习模型对性别、种族、年龄等特征的偏见。置信学习^[26]使用概率阈

值计数,来找出图片中的噪声标签,从而减少错误标签对模型的影响,提升图片分类模型的鲁棒性。差异训练^[27],通过记录样本在不同模型中迭代训练的损失并对其进行异常值检测来找出训练数据中的噪声标签,从而提升深度学习模型的鲁棒性。Suneja 等人^[28],在源代码理解任务中深度学习模型的数据收集,模型训练,预测分析三个阶段分别对影响模型鲁棒性的原因进行了讨论。

目前针对基于学习的源代码漏洞检测模型的鲁棒性研究工作还没有人进行过深层次的探讨。所以我们基于以上工作给出的启发,借鉴其中的思想和方法进行创新,尝试为解决基于深度学习的源码漏洞检测任务中影响模型鲁棒性的问题做出一些贡献。

我们在前期调研中的发现充分说明了现有的基于深度学习的漏洞检测模型鲁棒性不足。因此,本课题致力于探究影响基于深度学习的漏洞检测模型鲁棒性原因极其对应的优化方法。现阶段基于学习的漏洞检测模型存在的三个鲁棒性不足的表现:

- 1、漏洞检测模型在非同源的测试集上评估时,精度大幅下降。
- 2、漏洞检测模型所使用的数据集中包含大量噪声标签。
- 3、漏洞检测模型容易被对抗攻击成功绕过。

针对于我们前期调研发现的问题,我们给出本课题中鲁棒性的具体定义:源代码漏洞检测模型对于同性质代码数据无论在何种情况下预测结果的一致性,就称为源代码漏洞检测模型的鲁棒性。本课题致力于提出一个多维度的鲁棒性度量方法,对现有的基于深度学习的漏洞检测模型进行多维度的鲁棒性度量评估,并针对漏洞数据集中噪声样本较多的现状,提出基于集成学习与置信剪枝的漏洞数据集降噪方法,对现有的漏洞数据集进行降噪处理,以提高模型在正确样本上的学习权重。除此之外,本课题还针对漏洞数据集中样本单一的问题,提出了基于语义保护代码变化的数据增强方法,增加现有数据集的多样性,以达到数据增强的目的,通过样本降噪和数据增强来实现模型鲁棒性的提升。

本课题的研究目标为针对基于深度学习漏洞检测模型低鲁棒性的现状,给出多维度模型鲁棒性的度量方法,并从数据集入手提出优化模型鲁棒性的方法。效果就是将优化方法应用于现有基于深度学习的源码漏洞检测模型,提升其漏洞检测时的鲁棒性。拟解决的关键问题为,由于目前基于深度学习的源码漏洞检测模型低鲁棒性的现状,其本质上并没有学习到真正的漏洞样本特征,在提升鲁棒性的同时,目的是为了让模型学习到真正与漏洞相关的特征。

本课题的具体研究内容主要划分为以下三个部分:

- 1、提出基于集成学习与置信剪枝的漏洞数据集降噪方法,通过对数据集降噪来提升模型的鲁棒性。

- 2、提出基于语义保护代码变化的数据增强方法，通过数据增强来扩充数据集的多样性，从而提升模型的泛化能力，提升模型的鲁棒性。
- 3、提出一个多维度的鲁棒性度量方法，对现有的基于深度学习的漏洞检测模型进行鲁棒性度量评估，并证明优化方法的有效性。

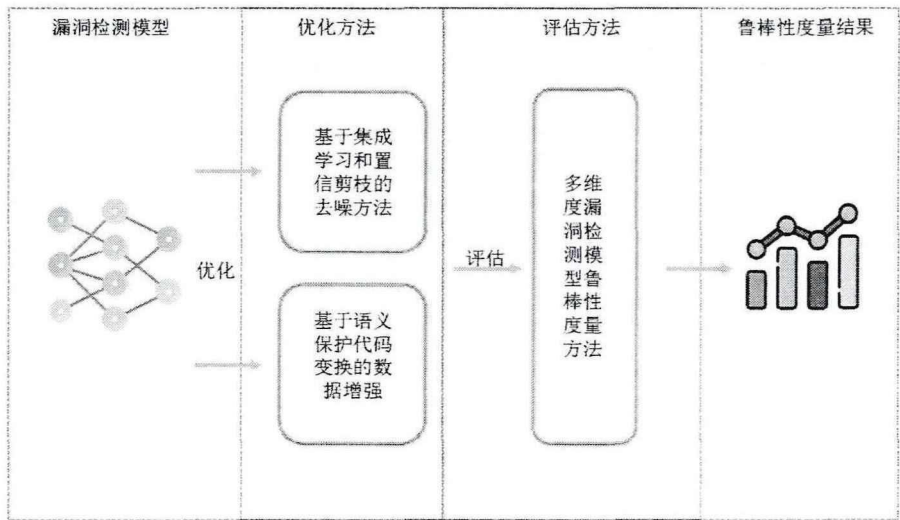


图 1-1 基于学习的源代码漏洞检测鲁棒性研究框架

本课题的整体框架如图 1-1 所示。通过多维度的漏洞检测模型鲁棒性评估模块来评估一个基于学习的漏洞检测模型是否足够鲁棒，若模型并不鲁棒，那么我们通过优化模块来进行模型优化，优化模块中包含两个优化方法，一是基于集成学习和置信剪枝的漏洞数据集降噪，二是基于语义保护的代码变换对数据集进行数据增强。通过优化方法优化过的模型，我们可以再次进行评估，以证明优化方法的有效性。

提出基于集成学习与置信剪枝的漏洞数据集降噪方法，通过对数据集降噪来提升模型的鲁棒性。基于初步调查研究并进行实验的结果来看，直接将置信学习用到源码漏洞检测上来效果不好，由于模型鲁棒性边界模糊的问题，虽然置信学习能够找到大量的噪声样本，但是找出噪声样本的同时也会去掉大量的正确样本，导致模型可学习的正确样本变少，又会导致鲁棒性降低。我们根据基于深度学习的漏洞检测模型的特点，不同模型有不同的鲁棒性边界，因此应对噪声的能力也不同，通过对多种不同网络结构的模型进行集成学习，分别在 k 折交叉验证过程中对样本给出预测，最后根据给不同模型权重分配来决定哪些样本被当作噪声样本进行剪枝处理，留下的样本即为正确样本，用于下一步训练任务。通过这种基于集成学习与置信剪枝的方法对数据集进行降噪，既能够处理数据集中大量的噪声样本，也能够保留足够训练的正确样本，从而提升模型训练数据的质量，提高模型的鲁棒性。

提出基于语义保护代码变化的数据增强方法，通过数据增强来扩充数据集的多样性，从而提升模型的泛化能力，提升模型的鲁棒性。经过调查研究以及一些实验结果的证明，被许多基于深度学习的漏洞检测方法用于训练，乃至评估的数据集 SARD 样本形式单一。甚至有研究表明，SARD 数据集中的漏洞类型 CWE119(内存缓存区内不恰当的操作限制)中，某些缓存区分配大小是固定的，且只存在于某一类中。原因是 SARD 数据集中大多数漏洞样本是根据预定义的漏洞模式自动生成的，用于学习和研究漏洞模式，因此与实际代码中的漏洞模式相差较远。因为实际代码中的漏洞模式，有大量的不相关信息，并且功能性代码给漏洞识别带来了挑战性。而漏洞模式单一的 SARD 只包含与漏洞相关的语句，因此用 SARD 来训练基于深度学习的漏洞检测模型，难免会有些偏差。因此我们提出使用语义保护的代码变换技术，构造多种不同的变换操作，对现有漏洞数据集增强，以增加漏洞样本的多样性，从而让模型能够学习到真正的漏洞语法或语义特征，而不是根据文本之间的相似性来判别是否包含漏洞。

提出一个多维度的鲁棒性度量方法，对现有的基于深度学习的漏洞检测模型进行鲁棒性度量评估。我们通过构造不同的训练数据及测试数据来从模型检测漏洞的性能、模型学习漏洞特征的能力、模型应对对抗样本时的鲁棒性和模型在非同源数据集上的鲁棒性四个维度来对模型进行系统的评估。

1.3 组织结构

本文共分为六个章节，包括以下内容：

第一章，绪论。该章节主要介绍基于深度学习的源码漏洞检测鲁棒性研究的选题背景和意义，讨论现有相关工作存在的问题，进一步引出本课题想要解决的问题、主要研究内容和研究方法。最后给出本文的组织结构安排。

第二章，相关研究及技术。该章节介绍本课题相关的背景知识，前人的研究工作和相关技术，包括源代码漏洞检测相关技术、抗噪声学习相关技术、对抗攻击及防护相关技术。

第三章，基于集成学习与置信剪枝的漏洞数据集去噪。该章节主要揭露现有漏洞数据集中存在噪声的普遍现象，并提出相应的去噪方法作为解决方案，并证明了该去噪方法的有效性。

第四章，基于语义保护代码变换的数据增强方法。该章节主要描述现阶段数据集样本模式单一，数据量不足的现状。并提出数据增强的方法来扩充数据集，并通过实验验证了数据增强的有效性。

第五章，多维度的漏洞检测模型鲁棒性评估方法。该章节着眼于漏洞检测领域缺乏的鲁棒性研究，提出了多维度漏洞检测模型鲁棒性度量方法，从多个角度度量模型鲁棒性，并通过度量方法证明了本课题提出的两个鲁棒性优化方法的有效性。

第六章，总结与展望。总结本文的研究内容，对研究工作存在的不足和未来的改进方向做出归纳。

第二章 相关研究及技术

本节将对研究基于深度学习的源码漏洞检测鲁棒性过程中的基本概念及使用的关键技术进行简要介绍和分析。包括源代码漏洞检测的相关技术、抗噪声学习、对抗攻击和代码变换技术。

2.1 源代码漏洞检测

源代码漏洞检测针对软件设计开发阶段,基于静态程序分析技术检测源代码中的漏洞,具有代码覆盖率高、漏报低的优点,但对已知漏洞的依赖性较大,误报较高。源代码漏洞检测方法主要包括基于中间表示的漏洞检测和基于逻辑推理的漏洞检测。基于中间表示的漏洞检测方法首先将源代码转换为有利于漏洞检测的中间表示,然后对中间表示进行分析,检查是否匹配预定义的某个漏洞规则,从而判断源程序中是否含有对应漏洞规则相关的漏洞。基于逻辑推理的漏洞检测方法将源代码进行形式化描述,然后利用数学推理、证明等方法验证形式化描述的一些性质,从而判断程序是否含有某种类型的漏洞。基于逻辑推理的漏洞检测方法由于以数学推理为基础,因此分析严格,结果可靠。但对于较大规模的程序,将代码进行形式化表示本身是一件非常困难的事情。基于中间表示的漏洞检测方法没有上述局限性,适用于分析较大规模程序,因此得到了更为广泛的应用。

2.1.1 静态分析技术

静态分析是指可以推断程序运行时状态而无需实际执行的自动方法。静态分析已用于检测可能导致程序过早终止或结果不明确的错误。以下是可以使用静态分析解决的漏洞的非详尽列表:

- 数组越界,访问超出其大小的数组元素。
- 空指针解引用,使用具有 null 值的指针。
- 内存泄漏,即程序没有回收临时使用的内存。
- 缓冲区下溢/上溢,即用于保存的数据值/内存的大小未得到考虑。
- 访问未初始化的变量/指针,即使用变量/指针而不分配值/内存。
- 无效的算术运算,例如,除以零。
- 非终止循环,例如,循环的退出条件永真。
- 非终止调用,即程序的控制流从不从函数调用返回。

静态分析技术包含以下主流方法:

语法模式匹配：该技术基于解析器对程序进行语法分析。解析器将程序源代码作为输入，并输出称为抽象语法树（AST）的数据结构。当用于漏洞查找时，该技术通过定义一组潜在危险或无效的程序构造来工作，然后在输入程序的 AST 中搜索这些构造的实例。句法模式匹配是静态分析中最快、最简单的技术。但它几乎不考虑程序正确性，并可能导致大量误报。

数据流分析：数据流分析通过构造程序的基于图的表示（称为控制流图），并为图的每个节点编写数据流方程来继续。然后重复求解这些方程，以计算每个节点处输入的输出，直到方程组稳定或到达固定点。所使用的主要数据流分析是到达定义（即，对变量的最新赋值）、实时变量分析（即，消除未使用的赋值）。在每个源代码位置，数据流分析记录了一组关于当前范围内所有变量的事实。除了要跟踪的一组实体之外，分析还为每个区块定义了一个“kills”集合和一个“gens”集合。“kills”集合描述了通过执行块中的语句失效的一组数据，而“gens”集合则描述了通过在块中执行语句而生成的一组数据。为了分析一个程序，分析工具从一组初始事实开始，并根据程序的每个语句的“kills”集合和“gens”集合依次更新它。

抽象解释：这是一种基于有序集上单调函数的程序语义近似理论。该理论背后的主要思想如下：一个具体的程序、它的具体领域和语义操作被一些抽象领域和抽象语义操作中的近似程序所取代。设 L 是一个有序集合，称为具体集合，设 L' 是另一个有序集，称为抽象集合。如果函数 α 将具体集合 L 中的元素 x 映射到抽象集合 L' 中的元素 $\alpha(x)$ ，则称为抽象函数。也就是说， L' 中的元素 $\alpha(x)$ 是 L 中 x 的抽象。如果函数 γ 将抽象集合 L' 中元素 x' 映射到具体集合 L 中元素 $\gamma(x')$ ，则称为具体化函数。设 L_1 、 L_2 、 L'_1 和 L'_2 为序集。具体语义 f 是从 L_1 到 L_2 的单调函数。从 L'_1 到 L'_2 的函数 f' 被认为是 f 的有效抽象，如果对于 L'_1 中的所有 x' ， $(f \circ \gamma)(x') \leq (\gamma \circ f')(x')$ 。应用抽象解释的主要挑战是抽象推理领域的设计。如果域太抽象，那么精度就会丢失，导致有效程序被拒绝。如果域太具体，则分析在计算上变得不可行。

基于约束的分析：基于约束的分析遍历程序，发出并求解描述程序属性的约束。这项技术分为两个步骤。第一步称为约束生成，从程序文本中生成约束，这些约束描述了程序所需的信息或行为。第二步是约束解析，它涉及通过计算期望的信息来解决约束。然后从这些解决方案中提取静态信息。该技术的一个关键特征是，用于约束求解的算法可以独立于所使用的最终约束系统来编写。

符号执行：在这种方法中，不向程序提供正常输入值，而使用表示任意值的符号。除了现在是输入值的符号公式之外，执行与正常执行相同。结果由程序计算的输出值被表示为输入符号值的函数。符号执行程序的状态包括程序变量的符

号值、路径条件和程序计数器。路径条件是符号输入上的无量词布尔公式，累积输入必须满足的约束，以便执行遵循特定的关联路径。符号执行树表示程序符号执行期间遵循的执行路径。树节点表示程序状态，它们通过程序转换连接。

2.1.2 动态分析技术

动态分析技术对系统的运行时行为进行分析。通常，动态分析包括记录程序的动态状态。这种动态状态也称为配置文件/跟踪。程序概要文件测量程序执行期间事件的发生。所测量的事件是程序局部的执行，如代码行、基本块、控制边缘、例程等。通常，动态分析技术包括以下阶段：1) 程序检测和配置文件/跟踪生成，2) 分析或监控。程序插桩是将附加语句插入程序以生成跟踪的过程。这些插入指令的语句在程序运行时同时执行。动态分析成功的根本挑战在于创建能够有效收集运行时信息的指令和分析。根据所提供的指令，将从一个或多个执行中收集所需信息用于分析目的。动态分析技术包含以下主流技术：

基于指令的分析：在该技术中，代码检测器用作预处理器，将检测代码插入目标程序。可以在编译过程的任何阶段添加此检测代码。基本上分三个阶段完成：源代码、二进制代码和字节码。源代码检测在使用源到源转换编译程序之前添加检测代码。它可以使用一些元编程框架，如 Proteus、DMS 等，自动插入额外的代码。这些元编程框架提供了一种编程语言，可用于定义对源代码的上下文敏感修改。转换程序被编译成执行源代码重写和插入的应用程序，源代码作为输入提供。将检测应用于源代码可以更容易地将跟踪功能与更高级别的、特定于领域的抽象相结合，这将最小化检测，因为附加代码的放置仅限于必要的内容。缺点是它依赖于目标语言，并且在处理特定的语言特性（如 C/C++ 预处理和句法变化）时也会出现问题。

基于 VM 分析：在该技术中，使用特定虚拟机提供的剖析和调试机制来执行动态分析。示例包括用于 .NET 框架的 Microsoft CLR Profiler 和用于 Java SDK 的 JPDA。这些分析程序可以深入了解程序的内部操作，特别是与内存和堆使用有关的操作。一种是使用 VM 的插件（实现为动态链接库）来捕获概要信息。这些插件称为评测代理，并以本机代码实现。这些插件通过接口访问虚拟机的分析服务。基于 VM 开发概要文件是很简单的，因为概要文件开发人员只需要实现 VM 提供的接口，而不必担心由于干扰正在运行的应用程序而导致的复杂性。

2.1.3 基于深度学习的漏洞检测技术

基于深度学习的漏洞检测技术旨在通过从训练模型从数据集中学习不同的漏洞模式来检测目标软件中的未知漏洞。最流行的基于深度学习的漏洞检测技术包括三个步骤：数据收集、模型构建和评估。首先，收集数据进行训练，并根据任

务目标和资源限制选择合适的模型。训练数据根据所选模型的输入格式进行预处理。然后训练模型并最小化损失函数。训练完成的模型是为了在实际的漏洞检测任务中使用。为了评价模型的有效性,对在模型未见过的测试用例上对模型的性能进行评估。

本小节描述了基于深度学习的漏洞检测技术方法、现有数据集、现有建模技术和评估程序的理论。

2.1.3.1 漏洞检测理论

基于深度学习的漏洞检测器从训练数据集中学习漏洞代码模式,其中代码样本被标记为漏洞或非漏洞。给定一个代码元素(x)和相应的漏洞/非漏洞标签(y),模型的目标是学习相对于模型参数(θ)最大化概率 $p(y|x)$ 的特征。

模型如何将 x_i 转换为 h_i 取决于模型的具体情况。 h_i 被转换为标量 $y \in [0,1]$,它表示代码样本 x_i 预测为漏洞的概率。

一般来说,这种转换和概率计算是通过模型中的前馈层和 softmax 层来实现的。通常,对于漏洞预测等二元分类任务,通过最小化交叉熵损失来学习最优模型参数。交叉熵损失惩罚模型的预测概率和实际概率(0 对应漏洞,1 对应非漏洞的样本)。

2.1.3.2 现有数据集

为了训练漏洞检测模型,通常需要一组带注释的代码,这些代码带有“漏洞”或“非漏洞”的标签。有漏洞的代码的数量应该足够大,以允许模型从中学习。研究人员使用了广泛的数据源来收集的数据。根据代码样本的收集方式和注释方式,将其分类为:

- 合成数据:漏洞的代码示例和注释是人为创建的。SATE IV Juliet 数据集和 SARD 属于这一类。这里的示例是使用已知的漏洞模式合成的。这些数据集最初是为评估传统的基于静态和动态分析的漏洞检测工具而设计的。

- 半合成数据:这里的代码或注释都是人为派生的。例如,由 Russell^[13]等人提出的 Draper 数据集,包含从开源存储库收集的函数,但使用静态分析器进行注释。SARD 和国家漏洞数据库(NVD)数据集的示例也来自生成代码。然而,它们经常被修改,以显示将它们与原始上下文隔离的漏洞。尽管这些数据集比合成数据集更复杂,但由于简化和隔离,它们并不能完全捕捉现实世界漏洞的复杂性。

- 真实数据:这里的代码和相应的漏洞注释都来自真实的来源。例如,Zhou 等人^[14]策划并设计了真实数据集,其中包括以往公开的漏洞及其来自四个开源项目的修复程序,其中两个是公开可用的。

2.1.3.3 现有建模技术

模型选择主要取决于想要学习的信息。基于深度学习的漏洞检测技术的流行选择是基于 Token 或基于图的模型,并且对输入数据(代码)进行相应的预处理。

•基于 Token 的模型:在基于 Token 的模型中,代码被视为一个 Token 序列。现有的基于 Token 的模型使用了不同的神经网络架构。例如, Li 等^[10]提出了基于双向长短期记忆(BSLTM)的模型, Russell 等^[13]提出了基于卷积神经网络(CNN)和随机森林的模型,并与基于循环神经网络(RNN)和 CNN 的漏洞预测基线模型进行了比较。对于这些相对简单的基于 Token 的模型,Token 序列长度是影响性能的一个重要因素,因为模型很难推理长序列。为了解决这个问题, VulDeePecker^[10]和 SySeVR^[11]提取了代码切片。切片背后的动机是,代码中的每一行对于漏洞检测都不一样重要。因此,不必让模型完全学习整个代码,只考虑从代码中“关键的点”提取的片段(例如, API 调用、数组索引、指针使用等)进行漏洞预测,而忽略其余不相关的代码部分。

•基于图的模型:这些模型将代码视为图,并包含不同的语法和语义依赖关系。漏洞检测可以使用不同类型的语法图抽象语法树(AST)和语义图(控制流图、数据流图、程序依赖图、Def-Use 链图等)。例如, Devign^[14]利用代码属性图(CPG)构建了他们的基于图的漏洞预测模型。CPG 是通过将不同的依赖边(即控制流、数据流、Def-Use 链图等)扩展到代码的抽象语法树(AST)来构造的。基于图的模型和基于 Token 的模型都必须处理词汇表爆炸的问题——代码中可能的标识符(变量、函数名、常量)的数量实际上是无限的,并且模型必须对这些标识符进行推理。解决此问题的常用方法是将 Token 替换为抽象名称。例如, VulDeePecker^[10]将大部分变量和函数名称替换为符号名称(VAR1, VAR2, FUNC1, FUNC2 等)。所有模型的期望输入都是实值向量,通常称为嵌入。有几种方法可以将 Token 嵌入到向量中。其中一种方法是使用与漏洞预测任务联合训练的嵌入层。另一种选择是使用外部词嵌入工具(例如 Word2Vec^[29])来创建每个标记的向量表示。VulDeePecker^[10]和 SySeVR^[11]使用 Word2Vec 将其符号转换为向量。一旦选择了模型,并对训练数据集进行了适当的预处理,就可以通过最小化损失函数来训练模型了。现有的方法大多通过最小化交叉熵损失的方法来优化模型。

2.1.3.4 评估程序

为了理解训练完成的模型在现实世界中检测漏洞的适用性,必须首先对其进行评估。在大多数情况下,训练好的模型是在测试集上进行评估的。测试样本经过与训练相同的预处理技术,然后模型预测这些预处理测试样本是否有漏洞的。这种评估方法给出了模型在现实世界中用于检测漏洞时的性能估计。虽然所有现有的方法都使用自己的评估数据集来报告它们的性能,但它并没有给出模型在现

实世界中的适用性的全面概述。我们只能从这种评估方法中了解到的是他们的方法是否适合自己的数据集。尽管在现实世界的项目中有一些有限的案例研究，这些案例研究并没有揭示假阳性和假阴性。假阳性和假阴性的数量与开发人员在漏洞修复中的努力直接相关，任何一个的数量过多都会阻碍开发人员使用模型。

2.2 抗噪声学习

随着最近大规模数据集的出现，深度神经网络（DNN）在许多机器学习任务中表现出令人印象深刻的性能，如计算机视觉、信息检索和语言处理。他们的成功取决于大量但仔细标记的数据的可用性，这些数据的获取既昂贵又耗时。一些非专业来源数据，如亚马逊的《机械图尔克》（Mechanical Turk）已被广泛用于降低高昂的标签成本；然而，使用这些来源通常会导致标签不可靠。此外，即使对于经验丰富的领域专家，数据标签也可能极其复杂。它们也可以通过标签翻转攻击进行对抗攻击。这种不可靠的标签被称为噪声标签，因为它们可能破坏真实样本的学习。据研究报告，真实世界数据集中损坏标签的比率在 8.0%至 38.5%之间。

在噪声标签存在的情况下，已知训练 DNN 容易受到噪声标签的影响，因为大量的模型参数使得 DNN 过度拟合到甚至具有学习任何复杂函数能力的损坏标签。Zhang 等人^[30]证明，DNN 可以很容易地拟合具有任何比例的损坏标签的整个训练数据集，这最终导致测试数据集的泛化性较差。虽然流行的正则化技术，如权重衰减、Dropout 和批量归一化已被广泛应用，但它们本身并不能完全克服过度拟合问题。即使所有上述正则化技术都被激活，基于干净和噪声数据训练的模型之间的测试精度差距仍然很大。

此外，标签噪声的精度下降被认为比其他噪声（如输入噪声）更有害。因此，在存在噪声标签的情况下实现良好的泛化能力是一个关键挑战。

目前抗噪声学习主要有以下几个方向的工作：

特殊模型结构：这种方法对有噪声数据集的标签转换矩阵进行了架构更改。这些变化包括在 softmax 层的顶部添加噪声自适应层，并设计新的专用架构。通过基于估计的标签转移概率修改 DNN 输出，得到的架构产生了改进的泛化。

正则化：正则化方法已被广泛研究，以提高机器学习社区中学习模型的可推广性。通过避免模型训练中的过度拟合，使用广泛使用的正则化技术（如数据增强、权重衰减、Dropout 和批量归一化）提高了对标签噪声的鲁棒性。这些正则化方法在中等噪声的数据上运行良好，但它们本身并不能充分提高测试精度，当噪声较大时，可能获得较差的泛化。因此，最近提出了更先进的正则化技术，当

与规范方法一起使用时,该技术进一步提高了对标签噪声的鲁棒性。这个系列的主要优点是它与其他方向合作的灵活性,因为它只需要简单的修改模型。

特殊损失函数:对于分类任务,分类交叉熵损失(CCE)由于其快速收敛和高泛化能力,是应用最广泛的损失函数。然而,在存在噪声标签的情况下,稳健的 MAE 表明,平均绝对误差(MAE)损失比 CCE 损失实现了更好的泛化,因为只有 MAE 损失满足上述条件。MAE 损失的一个限制是,当涉及复杂数据时,其泛化性能显著下降。因此,提出了广义交叉熵(GCE),以实现 MAE 和 CCE 损失的优点;GCE 损失是一类更一般的噪声鲁棒性损失,包括这两种损失。Amid 等人^[31]通过基于 Tsallis 散度引入两个因子,扩展了 GCE 损失。双调和损失^[32]引入了基于 Bregman 散度的 CE 损失的适当无偏概括。此外,受 KullbackLeibler 散度的对称性启发,通过将噪声容限项(即反向交叉熵损失)与标准 CCE 损失相结合,提出了对称交叉熵(SCE)。

损失调整:通过在更新 DNN 之前调整所有训练样本的损失,损失调整对于减少噪声标签的负面影响是有效的。根据其调整原理,与之相关的方法可分为三组:

- 1) 估计标签转移矩阵以校正正向或反向损失。
- 2) 对不同分类的样本进行重新加权。
- 3) 使用从噪声和预测标签的凸组合调整损失的标签翻新,以自动推断损失调整的最佳规则的元学习。

与新设计的鲁棒性损失函数不同,这一系列方法旨在使传统优化过程对标签噪声具有鲁棒性。因此,在训练过程中,会调整更新规则,以使标签噪声的负面影响最小化。通常,损失调整允许通过调整每个示例的损失来充分探索训练数据。然而,错误校正产生的误差是累积的,特别是当类的数量或错误标记的示例的数量很大时。

样本选择:为了避免任何错误的校正,样本选择从有噪声的训练数据集中选择正确的样本。在这种情况下,修改更新方程以使 DNN 对于有噪声的标签更加鲁棒。该训练计划动机良好,总体上运行良好,但仍存在错误选择导致的累积错误。因此,最近的方法通常利用多个 DNN 相互合作或运行多个训练 epoch。此外,为了从错误标记的示例中获益,许多工作将损失校正或半监督学习与样本选择策略相结合。

2.3 对抗攻击及防护

深度学习已经成为一个强大而有效的框架,可以应用于广泛而复杂的学习问题,这些问题以往很难用传统的机器学习技术来解决。深度学习的进步非常激进,以至于今天它已经能够超越人类水平的表现。因此,在最近的大多数日常应用中都广泛使用了深度学习技术。然而,使用精心制作的对抗性样本可能会危及高效的深度学习系统,这可能是人眼无法察觉的,但会导致模型错误分类输出。近年来,根据其威胁模型的不同类型的攻击利用这些漏洞来破坏深度学习系统,这些攻击有很高的动机。因此,为深度学习算法提供针对这些对抗的鲁棒性至关重要。然而,只有少数几种强有力的对策可以应用于所有类型的攻击场景,以设计一个强大的深度学习系统。

2.3.1 攻击手段

白盒攻击:在对机器学习模型的白盒攻击中,攻击者完全了解用于分类的模型(f) (例如,神经网络的类型以及层数)。攻击者拥有训练中使用的算法的信息(例如梯度下降优化),并可以访问训练数据分布(μ)。他还知道完全训练的模型架构的参数(θ)。攻击者利用这些信息分析模型可能脆弱的特征空间,即模型具有高错误率的特征空间。然后,通过使用对抗性样本制作方法改变输入来利用该模型。访问白盒攻击的内部模型权重对应于非常强的对抗性攻击。

黑盒攻击:相反,黑盒攻击假设不了解模型,并使用有关设置和先前输入的信息来利用模型。例如,在甲骨文攻击中,对手通过提供一系列精心制作的输入和观察输出来探索模型。黑盒攻击进一步细分为以下类别:

- 1) 非自适应黑盒攻击:对于目标模型(f),非自适应黑盒攻击只能访问模型的训练数据分布(μ)。然后,攻击为模型架构 f' 选择过程 Train' ,并在数据分布 μ 的样本上训练局部模型,以逼近目标分类器学习的模型。
- 2) 对于目标模型(f),自适应黑盒攻击没有关于训练过程的任何信息,但可以作为预言机访问目标模型。这种策略类似于密码学中的选择明文攻击。攻击者向目标模型发出自适应预言查询,并标记精心选择的数据集,即,对于任意选择的 x ,攻击者通过查询目标模型 f 来获得其标签 y 。然后,攻击者选择过程 Train' 和模型架构 f' ,以在查询目标模型获得的元组 (x, y) 上训练代理模型。

2.3.2 防御手段

对抗性训练:对抗性训练的主要目标是通过将对抗性样本注入训练集来提高模型的鲁棒性。对抗训练是一种标准的暴力训练方法,防御者只需生成大量对抗

示例，并在训练目标模型时扩充这些受干扰的数据。可以通过向模型提供合法数据和精心制作的数据，或者通过学习修改的目标函数来实现增强。对抗训练意味着对原始模型上制作的对抗样本进行训练。对于黑盒攻击，防御并不强大，攻击者在本地训练的替代模型上生成恶意示例。此外，对抗性训练可以通过两步攻击轻松绕过，其中随机扰动首先应用于实例，然后对其进行任何传统攻击。

梯度隐藏：基于梯度的攻击和使用 FGSM 等对抗性训练方法的攻击的自然防御可能包括向对手隐藏关于模型梯度的信息。例如，如果模型是不可微的（例如决策树、最近邻分类器或随机森林），则基于梯度的攻击将变得无效。

2.4 本章小结

本章节介绍了与课题相关的背景知识、研究工作和技术。首先介绍了源代码漏洞检测技术相关的背景知识，包括静态分析技术、动态分析技术和课题研究中最关键的基于深度学习的漏洞检测技术。其次介绍了在本课题优化模型鲁棒性方面用到的抗噪声学习方法。最后讨论了影响模型鲁棒性的对抗攻击所涉及到的关键技术。

第三章 基于集成学习与置信剪枝的漏洞数据集去噪

3.1 问题描述

3.1.1 噪声的普遍性

为了探究基于深度学习的漏洞检测器中使用的数据集中标签错误的普遍性，我们首先参考了最先进的漏洞检测工作以及它们使用的漏洞数据集。我们还从漏洞收集过程的角度强调了检测模型训练数据集中存在的问题，这与样本标注错误的原因密切相关。

3.1.1.1 基于学习的漏洞检测方法

表 3-1 基于学习的漏洞检测方法

方法	数据集	标注方式	检测粒度
DeepWukong ^[16]	SARD 和 Real-world data	手动	语句
VulDeePecker ^[10]	SARD 和 NVD	Code-diff+ 手动	语句
SySeVr ^[11]	SARD 和 NVD	Code-diff+ 手动	语句
MulVulDeePecker ^[12]	SARD 和 NVD	Code-diff	语句
VGDetector ^[15]	SARD	自动	函数
Devign ^[14]	Real-world data	Code-diff	函数
Russell ^[13]	SARD 和 Real-world data	静态分析工具	函数
IVDetect ^[33]	Big-Vul, REVEAL, FFMpeg+Qemu	Code-diff	函数
ReVeal ^[9]	REVEAL	Code-diff	函数
FUNDED ^[34]	SARD , NVD 和 Real-world data	手动	函数

本课题已经在第一章中详细描述了表 3-1 中所有前沿的漏洞检测方法所使用的数据集中存在噪声标签的普遍现象。接下来将从漏洞数据收集过程的角度来阐述这个问题。

3.1.1.2 收集与漏洞修复相关的 Commits

表 3-2 漏洞数据收集方法

数据集	标注方式	开源组件	Commits 是否包含噪声
D2A ^[35]	Infer 和差异分析	Bug Trace Commit	√
Big-Vul ^[36]	CVE 映射到 Commit	Commits 和 CVE	√
VulinOss ^[37]	CVE 映射到 Commit	Patches	√
REVEAL ^[9]	爬取漏洞库及 NVD	函数级别的源码	√
FFMPeg+Qemu ^[14]	正则+手动分析	函数级别的源码	×
Cross-Vul ^[38]	CVE 映射到 Commit	文件级别的源码	√
Wang ^[39]	基于学习的方法	Commits	√
Sabetta ^[40]	基于学习的方法	Commits	√
Zhou ^[41]	基于学习的方法	Commits	√
VccFinder ^[42]	基于学习的方法	Commits	√
FUNDED ^[34]	基于学习的方法	Commits	√

如表 3-2 所示，在参考了多个最先进的基于学习的漏洞检测方法之后，我们探索了多个专门用于构建现实世界漏洞数据集的工作。我们总结了这些工作在收集漏洞数据的过程中可能暴露的问题。

收集真实源代码漏洞数据集的传统方法^[9,14,35,36,37,43]包括以下三个步骤：

- (1)通过关键字匹配从公开可用的补丁数据库或漏洞数据库中收集与漏洞修复相关的补丁。
- (2)使用静态分析工具或手动进一步过滤掉与 bug 修复无关的补丁。（有些方法不包括这一步）。
- (3)根据补丁修复前后的代码差异，认为补丁修改的函数或切片是有漏洞的，否则为无漏洞的。

这些基于规则的方法受到静态分析工具的规则和功能的限制，通常会产生更多的错误标签。除了上述传统的基于规则的漏洞相关提交收集方法外，我们还参考了五种基于学习的工作^[34,39,41,42,44]，它们收集漏洞相关提交的过程可以描述为以下五个步骤：

- (1)收集漏洞相关 commits 作为 ground-truth。

- (2)从 commits 中提取特征, 包括代码差异和 commits msg。
- (3)将提取的特征嵌入到向量中。
- (4)训练一个二元分类模型(机器学习模型或深度学习模型)来学习 commits 的特征。
- (5)预测未知 commits 是否与漏洞相关。

尽管基于学习的方法比基于规则的方法更有效地识别与漏洞相关的 commits, 但它们仍然会产生更多的假阳性。因此, 基于学习的方法收集的漏洞相关 commits 很难应用于漏洞检测模型的训练。

综上, 无论是基于学习的方法收集的 commits, 还是基于规则的方法收集的 commits, 当这些与漏洞相关的 commits 应用于漏洞检测模型的训练时, 总会产生一些错误标签。

3.1.2 漏洞数据集中引入噪声标签的原因

3.1.2.1 漏洞数据集中噪声的定义

我们将噪声定义为用于训练模型的漏洞样本的错误标签, 它可以是函数级的, 也可以是语句级的。将漏洞样本(1)标记为非漏洞样本(0), 或将非漏洞样本(0)标记为漏洞样本(1)称为噪声样本, 尽管后者可能更常见。

3.1.2.2 噪声起因

在本小节中, 我们将噪声的来源系统地总结为两个阶段, 与漏洞相关的 commits 收集阶段和漏洞样本标记阶段。

3.1.2.2.1 commits 收集阶段

第一个关键阶段是收集与漏洞相关的 commits。这些工作即 [9,14,34,35,36,37,39,41,42,43,44] 无论是基于学习的方法还是基于规则的方法, 都包括与漏洞相关的 commits 的收集阶段。基于实际技术条件, 模型预测不能 100%正确, 因此基于学习的方法收集的 commits 必然会包含与漏洞修复无关的 commits。

基于规则的方法包括两种。第一种方法从发布漏洞的网站 (CVE 或 NVD) 抓取一些漏洞实体, 然后将它们映射到相应的 commits 版本。但是, 在某些情况下, 这些网站提供的参考链接可能指向与漏洞修复版本无关的 commits, 这可能导致收集的 commits 与漏洞修复无关。

研究人员提出了第二种方法来解决上述问题并收集与漏洞修复相关的 commits。它们使用关键字来匹配 commits msg 中包含安全相关问题的字符串。不幸的是, 与漏洞无关的 commits 并没有被完美地消除, 因为代码托管平台没有规定统一的 commit msg 格式, 这导致不同项目的 commit msg 各不相同。随之而

来的现象是，一些非标准的 commit msg 描述将导致使用基于规则的方法来收集一些与漏洞修复无关的 commit。

为了证明这些作品收集的漏洞相关提交中确实存在不相关的 commits，我们分别从 D2A^[35]、Big-Vul^[36]和 Cross-Vul^[43]发布的数据集中随机抽取 100 条数据，手动检查它们是否与漏洞相关。我们从采样数据中删除带有重复 id 的 commits，因为这些数据指向相同的修复版本。判断 commit 是否与漏洞相关的标准是由此 commit 版本修改的代码行是否包含对于某个漏洞的修复。

表 3-3 手动检查 commits 的结果

数据集	样本数量	唯一 commit 数量	漏洞相关数量	漏洞相关率
D2A ^[35]	100	79	59	74.7%
Big-Vul ^[36]	100	93	68	73.1%
Cross-Vul ^[43]	100	100	93	93%

手动检查的结果如表 3-3 所示。D2A 在采样数据中包含更多的重复 commits，而 BigVul 和 Cross-Vul 的重复相对较少。就漏洞相关 commits 的比例而言，D2A 和 BigVul 的漏洞相关比例相似，因为两个数据集中的大多数 commits 都涉及对多个文件的修改，因此更有可能包括与功能性更新相关的 commits。Cross-Vul 中的 commits 与漏洞最相关，因为这些 commits 通常只修改一个文件，并在 commits msg 中包含 CVE-ID。这只是一个随机抽样的结果，用来证明这些工作收集的数据包含与漏洞无关 commit。事实证明，在最坏的情况下，可能有大约 30%的 commit 会导致数据集中引入噪声标签。请注意，在第一阶段中，我们只确认漏洞 commits 收集是标签噪声的主要来源之一。

3.1.2.2.2 漏洞样本标记阶段

引入噪声标签的另一个阶段是漏洞样本标记阶段。在这一阶段，我们重点研究了各种漏洞检测方法在标注训练样本时可能会引入噪声标签的原因，这可能包括以下两个原因：

首先，我们解释了与数据集本身相关的原因。从表 3-1 可以看出，很多漏洞检测方法都使用 SARD 进行训练。但是，与其他自动标记数据集相比 SARD 是一个具有相对可靠标签的合成数据集。但是，我们发现 SARD 提供的 XML 文件错误地将一些代码文件的第 0 行标注为漏洞代码行，从而导致一些包含漏洞的代码行被标记为非漏洞。这将导致一些语句级粒度和函数级粒度漏洞检测方法将漏洞样本(即标签 1)标记为非漏洞样本(即标签 0)。为了创建干净的基准数据集进行评估，我们增加了预处理步骤，系统地检查 SARD 中不同漏洞类型中的这种错

误标记问题，并相应地去除，如表 3-4 所示。总的来说，错误标注只占总样本的一小部分。

表 3-4 SARD 中漏洞行标注为 0 的文件统计

CWE-ID	漏洞文件数量	漏洞行标注为 0 的文件数量	占比
CWE020	18335	928	5.1%
CWE119	7182	918	12.8%
CWE787	1535	25	1.6%
CWE400	5772	5	0.1%
CWE125	1433	0	0%
CWE190	2164	0	0%

然后我们讨论了样本标记方法可能引入的噪声标签。很多基于程序切片的漏洞检测方法都会对数据集进行预处理，如 VulDeePecker、SySeVr、DeepWukong 等。它们将在程序切片之后表示为代码切片。代码切片是程序切片之后的代码片段。符号化将具有相同语义的代码片段表示为统一形式。正如 ReVeal^[9]所发现的，符号化后的代码片段中包含大量的重复样本和标签冲突样本。标签冲突是指具有相同文本内容的样本同时被标记为正(1)和负(0)，这会引入噪声标签。在这三种方法中，只有 DeepWukong 声明他们做了重复数据删除。标签冲突的根本原因是源代码模式过于简单。数据集中的一些漏洞修复只做了很小的修改，所以在符号化之后会添加重复的样本，甚至会出现标签冲突。

另一种引入噪声标签的标记方法是根据 commits 提供的 code-diff 标注训练样本。根据我们之前的发现，许多漏洞检测方法根据 commits 提供的 code-diff 来标记样本，code-diff 记录了该 commit 修改、删除和添加的代码行。这些方法简单地将更新前的版本中包含删除或修改行的函数或语句标记为有漏洞(1)，更新的版本中对应的函数或语句标记为非漏洞(0)，未修改的函数或语句也标记为非漏洞(0)。

如果将功能更新修改的代码也被标记为有漏洞，则会引入噪声标签，因为 commit 通常不仅包括漏洞修复，还包括功能更新(与漏洞无关)。

为此，我们从 GitHub 上的一个流行项目 Redis 收集了 100 条 commit。这 100 条 commit 的 commit msg 都包含 CVE-ID，因此我们认为这些 commit 都与漏洞修复有关。我们根据 code-diff 检查了每个 commit 更改的源代码文件和函数个数。100 次 commit 共修改了 139 个 CPP 文件和 198 个函数，其中 145 个函数存在漏洞。这表明，即使收集的 commit 都与漏洞修复相关，根据 code-diff 标记漏洞函数也会产生噪声样本，更不用说包含一些 commit 与漏洞修复无关。

上述工作充分证明了目前使用的数据集是极其不可靠的。数据集的标签质量决定了漏洞检测模型能否学习到真正与漏洞相关的特征。因此，本小节的目的是通过减少数据集中的噪声标签来提高数据集的标签质量。

3.2 去噪算法原理

3.2.1 置信学习

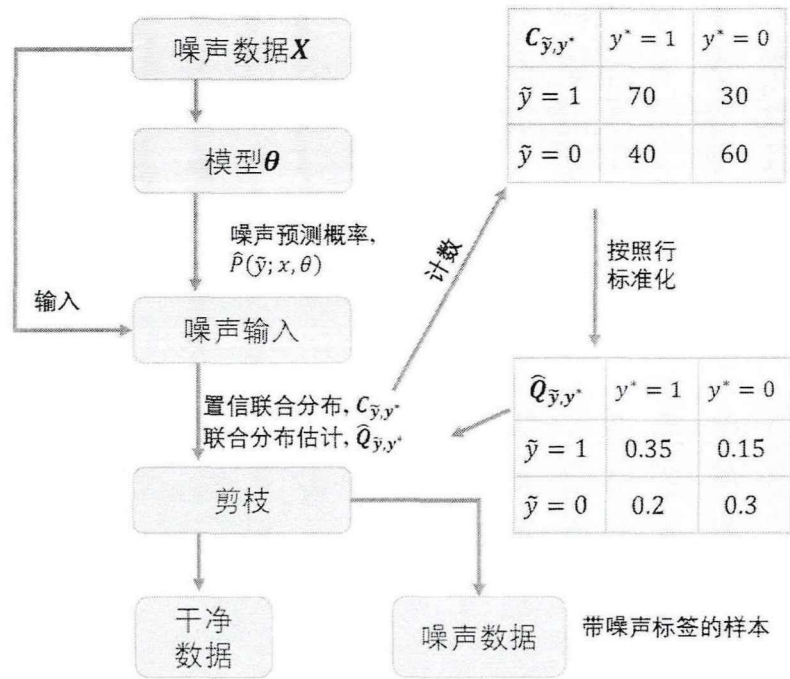


图 3-1 置信学习 (CL)

CL 基于剪枝噪声数据、用计数来估计噪声、通过对样本排序来进行置信训练的原则工作。CL 通过估计噪声标签 $\tilde{y}$ 和真实标签 y^* ，如图 3-1，CL 要求两部分输入：(1) 样本外预测概率 $\hat{p}_{k,1}$ 。(2) 噪声标签 $\tilde{y}_k$ 。样本外预测概率是模型 θ 对样本 x_k 的包外估计。在 CL 中，该步骤通过交叉验证实现。通过 k 折交叉验证，模型对每个样本给出分类为 1 的概率和分类为 0 的概率。然后 CL 计算置信联合分布，这是一种统计数据结构，通过直接剪枝策略来查找有噪声的标签。

$$t_j = \frac{1}{\text{Count}(X_{\tilde{y}=j})} \sum_{x \in X_{\tilde{y}=j}} \hat{p}(\tilde{y} = j; x, \theta) \tag{3-1}$$

$$\widehat{Q_{\tilde{y}=i, y^*=j}} = \frac{\frac{C_{\tilde{y}=i, y^*=j}}{\sum_{j \in [0,1]} C_{\tilde{y}=i, y^*=j}} \cdot \text{Count}(X_{\tilde{y}=i})}{\sum_{i,j \in [0,1]} \left(\frac{C_{\tilde{y}=i, y^*=j}}{\sum_{j' \in [0,1]} C_{\tilde{y}=i, y^*=j'}} \cdot \text{Count}(X_{\tilde{y}=i}) \right)} \tag{3-2}$$

给定置信联合矩阵 $C_{\tilde{y},y^*}$ 可以根据公式 3-2 得出 $Q_{\tilde{y},y^*}$ 。分子进行校准，以使行总和和标签分布相匹配。分母进行校准，使概率分布总和为 1。给定 $Q_{\tilde{y},y^*}$ ，我们使用 $n \cdot Q_{\tilde{y},y^*}$ 来估计 $\text{Count}(X_{\tilde{y}=1,y^*=1})$ ，并通过概率排序来剪枝。对于 $Q_{\tilde{y},y^*}$ 中每个非对角线元素 $Q_{\tilde{y},y^*}, i \neq j$ ，我们从 $x \in X_{\tilde{y}=i}$ 中选择符合 $\widehat{p_{x,\tilde{y}=j}} - \widehat{p_{x,\tilde{y}=1}}$ 条件的 $n \cdot Q_{\tilde{y},y^*}$ 个样本作为噪声样本。

3.2.2 差异训练

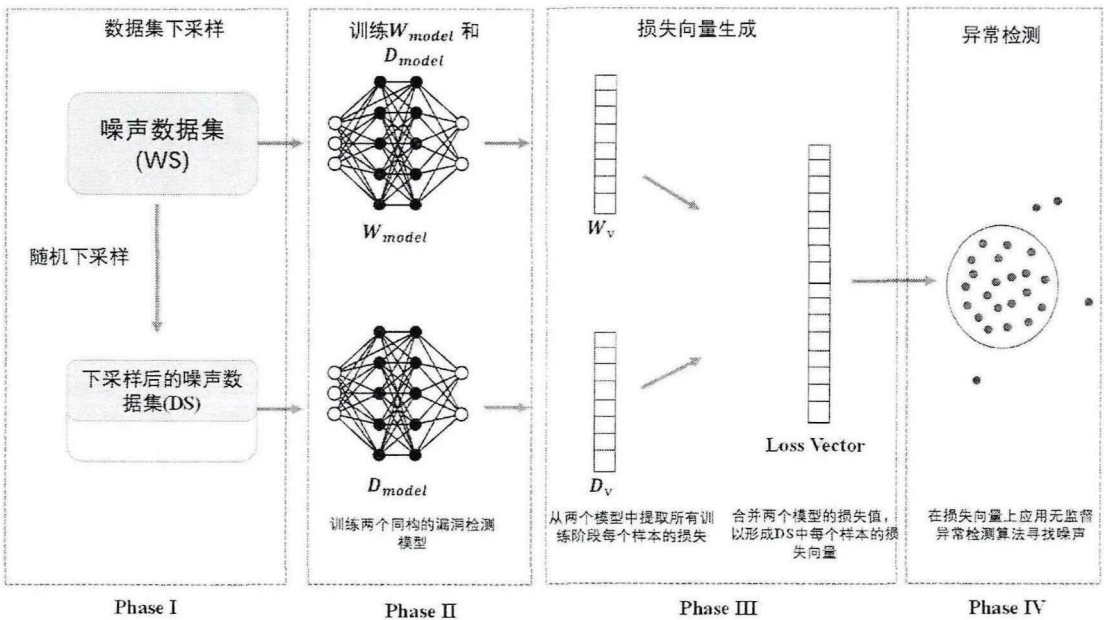


图 3-2 差异训练 (DT) 原理

如图 3-2 所示，DT 的整个过程包括多次迭代。在每次迭代中，需要训练两个同构模型 W_{model} 和 D_{model} ，其中 W_{model} 使用全部训练数据集进行训练， D_{model} 使用整个训练数据集的随机下采样子集来训练。DT 在训练期间使用两个相同的深度学习分类模型的所有中间状态，并为每个下采样样本生成噪声检测特征向量。然后，DT 在所有噪声检测特征向量上应用一组离群值检测算法，以通过多数投票决定哪些样本是噪声样本，最终在下一轮迭代前去除给定训练数据集的噪声标签。

3.2.3 基于集成学习和置信剪枝的漏洞数据集去噪

置信学习 (CL) 是为了在图像分类领域中识别图像数据集中错误标注的图片而提出的一种基于分类器置信度阈值的一种去噪声的方法，这种方法更适合于多标签多分类且数据量丰富的图像分类领域。它默认数据集中大部分标签都是标注正确的，并且当前基于深度学习的图像分类技术已经达到了超越人眼的水平，因此图像分类模型给出的置信度有相当大的说服力和可解释性。因此 CL 在图像数据集中能够以极低的误报给出精度极高的噪声标签反馈。

相比图像分类领域，源代码漏洞检测领域中，漏洞检测任务是一个非黑即白的二分类任务，且当前的基于学习的漏洞检测模型给出的置信度，不足以让人信任，因此，在去噪结果中，任意一个漏洞检测模型应用 CL 去噪时会大量的正常样本（非噪声）误判为噪声样本，这类被误判的噪声样本通常是分类边界较为模糊且较为复杂的样本，使用单个模型不足以拟合此类样本的分类边界。

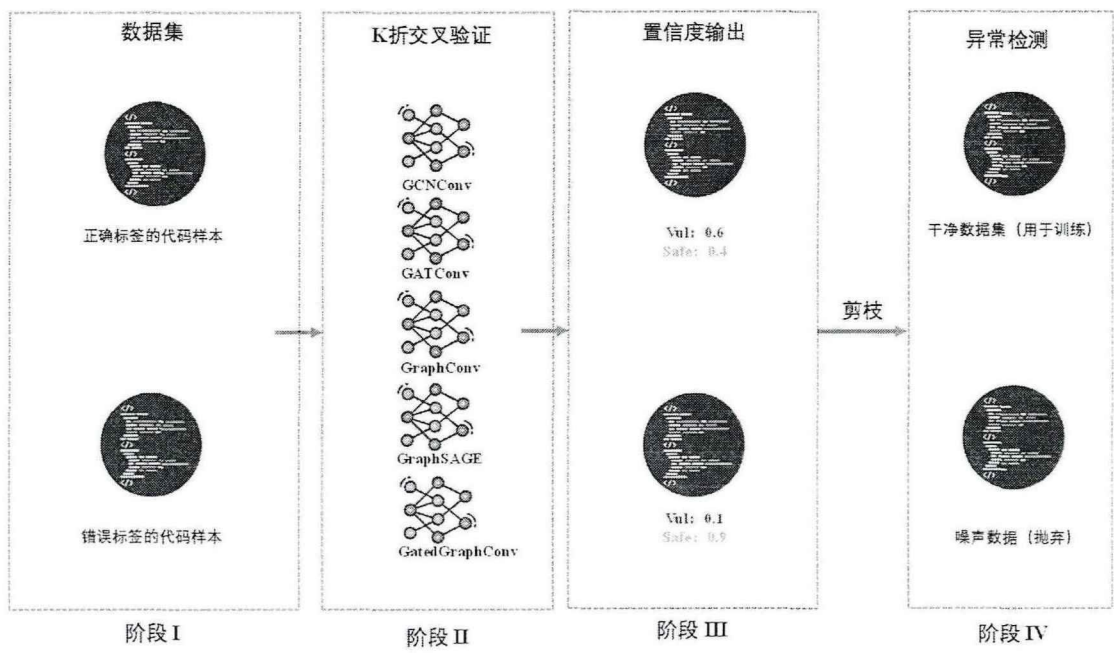


图 3-3 基于集成学习和置信剪枝的去噪原理

因此，我们提出使用集成学习的方式，使用多个异构的网络模型来多角度学习漏洞数据集的样本特征，这样多个模型对于复杂样本的分类边界可以通过多数投票来拟合，从而降低了被误判的风险，这样就大大提升了集成学习模型给出置信度的可信性。再结合 CL 的技术，我们可以更加精准的判别漏洞数据集中的噪声，从而提升数据集的标签质量，解决模型鲁棒性不足的问题。基于集成学习与置信剪枝的原理如图 3-3 所示。我们在模型设计时采用加权投票的集成学习模式将多个异构的 GNN 模块（GCNConv、GATConv、GraphConv、GraphSAGE 和 GatedGraphConv）集成来融合多个角度的特征提取结果，以达到最鲁棒的决策边界，再通过 CL 的去噪框架来输出最终的噪声样本。

3.3 实验设置

3.3.1 数据集选择

我们选择 SARD 作为合成数据集，FFMPeg+Qemu 作为真实数据集，因为它们的标签足够可靠。特别是 SARD 的标签相对于其他自动标记的数据集更为准

确, FFMpeg+Qemu 的标签是经过人工检查和验证的。为了进一步保证后续所有实验的可靠性, 我们在使用 SARD 作为 BaseLine 数据集之前采取了预处理步骤, 以过滤掉 SARD 中可能标记错误的源代码文件。

3.3.2 漏洞检测 BaseLine

CL 和 DT 都可以应用于任何适合 SARD 的漏洞检测模型, 因为它们是模型无关的去噪方法。我们选择 VulDeePecker、SySeVr 和 DeepWukong 来评估 CL 和 DT 应用不同漏洞检测模型的有效性。我们使用这三个模型有以下两个原因:

- (1) 它们都在训练阶段使用 SARD 数据集。
- (2) 它们都是语句级检测粒度。

在这三种方法中, 通过程序切片和代码嵌入, 使用训练完成的模型对目标输入程序生成的嵌入向量进行分类。最后, 模型可以预测一个从未见过的样本是有漏洞的还是安全的。

对于这三种方法在程序切片、代码嵌入和模型训练三个阶段的程序实现和参数设置, 我们在严格按照原始论文中描述进行, 以保证实验结果的正确性和公平性。

3.3.3 数据标注

我们分别使用 VulDeePecker、SySeVr 和 DeepWukong 中的方法生成 Code Gadgets、SeVCs 和 XFGs。对于 SARD, 从“good”程序中提取的样本被标记为无漏洞(0), 从“bad”或“mixed”程序中提取的样本如果包含至少一个漏洞语句, 则标记为有漏洞的(1), 否则标记为无漏洞的(0)。

对于 FFMpeg+Qemu, 包含被 commit 修改或删除的语句的样本被标记为有漏洞的(1), 否则为无漏洞的(0)。

3.3.4 生成噪声标签

CL 和 DT 都依赖于一个假设, 即数据集中的大多数样本标签都是正确的。在第 3.1 节中描述真实世界漏洞数据集的噪声标签时, 我们大致估计了数据集中噪声标签的比例。在本文中, 我们设置了实验来评估 CL 和 DT 在三个噪声水平(10%, 20%和 30%)下的有效性。

样本标注完成之后, 非漏洞样本数量远大于漏洞样本数量。如果我们在此基础上按比例设置噪声, 标记为非漏洞(0)的样本的数量远大于漏洞(1)样本的数量。因此, 我们通过随机抽样非漏洞(0)样本的数量到漏洞(1)样本数量的两倍来进行下采样。

在下采样之后我们通过如下规则来进行噪声标签的生成：给定一个噪声率 R_noise (10%, 20%, 30%), 我们以概率 R_noise 随机反转数据集中的每一个样本标签, 成功反转的就是我们要生成的样本标签。实验中所使用的样本数量和反转标签的数量如表 3-5 所示。

在我们的实验中, 我们只在训练集中产生有噪声的标签, 同时保留原始且干净的测试集。通过这种方式, 我们可以探索噪声标签对模型预测的真实影响, 以及去噪方法在提高模型预测能力方面的有效性。否则, 有噪声标签的测试集会在评价中产生偏差, 从而无法得出准确可靠的结论。

表 3-5 样本及噪声样本数量

CWE- ID_ R_noise	样本数量			翻转样本数量		
	XFG	SeVC	Code Gadget	XFG	SeVC	Code Gadget
CWE020_10%	107238	40974	13512	10723	4097	1351
CWE020_20%	107238	40974	13512	21447	8194	2702
CWE020_30%	107238	40974	13512	32171	12292	4053
CWE119_10%	64524	29157	12033	6452	2915	1203
CWE119_20%	64524	29157	12033	12904	5831	2406
CWE119_30%	64524	29157	12033	19357	8747	3609
CWE787_10%	43257	15801	6486	4325	1580	648
CWE787_20%	43257	15801	6486	8651	3160	1297
CWE787_30%	43257	15801	6486	12977	4740	1945
CWE400_10%	20031	13965	6027	2003	1396	602
CWE400_20%	20031	13965	6027	4006	2793	1205
CWE400_30%	20031	13965	6027	6009	4189	1808
CWE125_10%	11031	8757	3651	1103	875	365
CWE125_20%	11031	8757	3651	2206	1751	730
CWE125_30%	11031	8757	3651	3309	2627	1095
CWE190_10%	7665	5391	2298	766	531	229
CWE190_20%	7665	5391	2298	1533	1063	459
CWE190_30%	7665	5391	2298	2299	1595	689

3.3.5 硬件标准

我们的实验是在一台运行 Ubuntu Linux、NVIDIA GeForce GTX GM200 GPU 和 Intel(R) Xeon(R) CPU E5-2603 v4、1.70GHz 和 64GB 内存的服务器上进行的。

3.3.6 CL 和 DT 的实现

对于 CL 的实现，我们使用 CL 提供的 Python 包 cleanlab。CL 基于 k 折交叉验证计算包外估计，本文设 $k=5$ 。对于 DT 的实现，我们使用 Pytorch 来实现整个迭代过程。对于离群值检测阶段，与 DT^[27]相同，我们使用 13 种离群值检测方法和相应的参数设置。一旦在最后几次迭代中估计噪声比的波动小于某个阈值，我们就停止迭代。

3.3.7 评估指标

为了便于区分，我们分别引入了漏洞检测指标和噪声检测指标。

漏洞检测度量：对于漏洞检测结果，TP 是检测到真正漏洞样本的数量。FP 是检测结果是漏洞但实际上是非漏洞的样本数量。TN 是检测到的真正非漏洞的样本数量。FN 是检测结果非漏洞样本但实际上为漏洞样本的数量。我们使用 F1-Score 以衡量漏洞检测方法的有效性。

噪声检测度量：对于噪声检测结果，TP_n 是检测到真正噪声样本的数量。FP_n 是检测结果是噪声但实际上是非噪声的样本数量。TN_n 是检测到的真正非噪声的样本数量。FN_n 是检测结果是非噪声样本但实际上噪声样本的数量。我们使用 F1-Score_n 以衡量噪声检测方法的有效性。

3.4 实验及结论

为了评估噪声标签的影响和去噪方法的有效性，我们使用漏洞检测 BaseLine 进行了五个实验。总体而言，我们的评估针对以下方面：

- 1) 数据集中的噪声标签对漏洞检测器性能的影响。
- 2) 去噪方法的有效性。
- 3) 漏洞检测器性能对去噪方法有效性的影响。
- 4) 应用去噪方法后，漏洞检测器性能提高的可能性。

3.4.1 噪声对漏洞检测器的影响

在揭示了最先进的基于学习的漏洞检测方法所使用的数据集中普遍存在的噪声标签之后，我们进一步探讨了这一研究中的两个问题：1) 模型在干净测试集中的性能是否会受到训练集中的噪声标签的影响？2) 训练集中的噪声是否会

影响模型评估的排名？为了回答这两个问题，我们评估三种漏洞检测模型（DeepWukong、SySeVr 和 VulDeePecker）针对六种漏洞类型（CWE020、CWE119、CWE787、CWE400、CWE125 和 CWE190）和四种噪声级别（0%、10%、20%和 30%）上的 F1-Score。注意，如第 3.3.1 节所述，我们过滤掉漏洞行号标记为 0 的代码文件，以确保数据集的清洁度和评估可靠性。如第 3.3.4 节所述，噪声标签仅以可控的方式生成并添加到训练数据中。

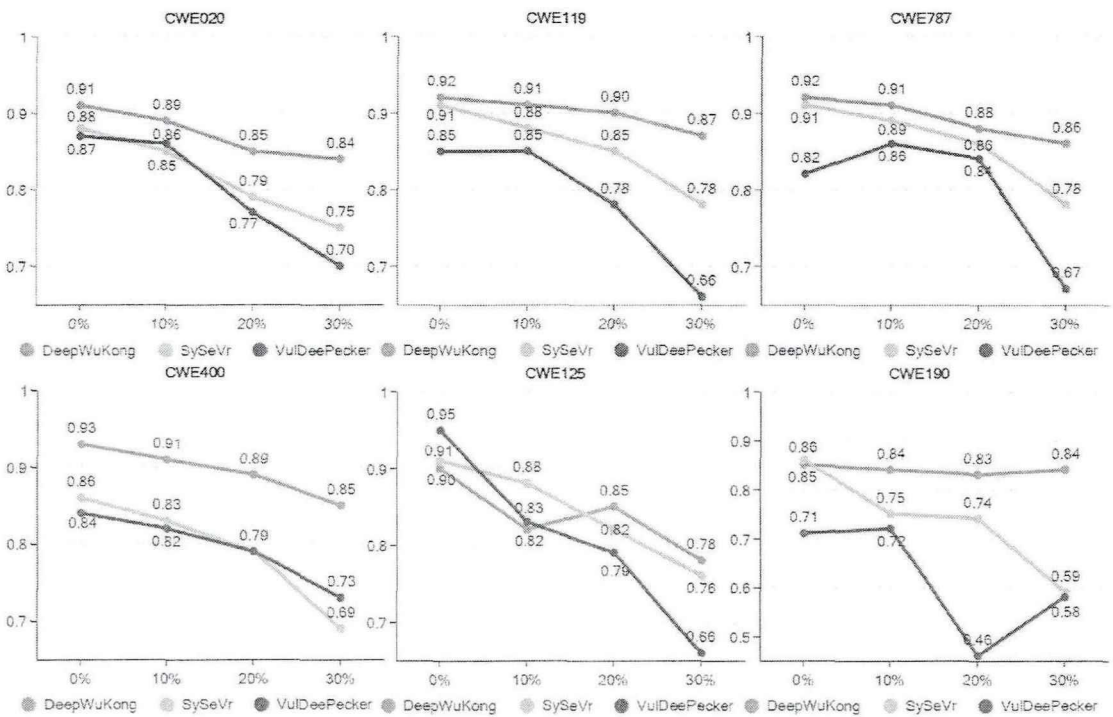


图 3-4 三个漏洞检测模型分别在四个噪声等级上的 F1-Socre

表 3-6 三个漏洞检测方法分别在不同噪声等级上的 F1-Score

漏洞检测方法	0%噪声	10%噪声	20%噪声	30%噪声	方差
DeepWukong	0.905	0.878	0.867	0.841	0.00069
SySeVr	0.888	0.846	0.809	0.726	0.00477
VulDeePecker	0.840	0.823	0.738	0.666	0.00651

结果：如图 3-4 所示，随着噪声数量的增加，我们观察到大多数漏洞检测模型的性能加速下降。有最明显的表现的是 CWE119。DeepWukong、SySeVr 和 VulDeePecker 的 F1-Score 都随着噪声等级的上升有明显的下降，但是 VulDeePecker 下降的最多，从 0.85 下降到 0.66。比较三个漏洞检测模型在噪声中的学习性能，我们可以从图中看到，DeepWukong 受噪声影响最小，其次是 SySeVr，VulDeePecker 受噪声影响最大。此外，我们计算三个漏洞检测模型在六

种漏洞类型上的 F1-Score 平均值，计算四种噪声水平下的方差，并将其记录在表 3-6 中。从三个模型的方差中，我们可以看到 DeepWukong 的方差最小，因此它受噪声的影响最小。此外，从 CWE020、CWE125 和 CWE190 的折线图中，我们可以看到三个漏洞检测模型的折线有交叉点，这表明它们的排名在噪声的影响下发生了变化。

分析：漏洞检测模型受噪声影响的背后原因是深度学习模型在学习过程中会拟合噪声样本，从而影响了它们对无噪声测试数据的预测能力。同时，我们还注意到 CWE125 和 CWE190 在噪声水平分别为 20%和 30%时的性能异常上升。这可能是因为与其他漏洞类型相比，它们的数据样本数量相对较少，如表 1 所示。因此，随着有噪声样本的比例增加，用于学习的样本（无噪声）减少，这可能导致训练模型的性能有一个无效的提升。就模型从噪声中学习的能力而言，它也与模型拟合数据集的能力呈正相关。模型拟合数据集的能力越强，模型就越能从包含噪声标签的数据集中学习正确的知识。拟合数据集能力不强的模型，它的决策边界会更容易受到噪声样本的影响。此外，在不同的噪声水平下，三个模型的排序顺序也发生了变化，这表明数据集中存在噪声使得模型的评估不可靠。

结论：漏洞检测模型的性能受到数据集中噪声的影响，其中噪声的影响与噪声水平成比例。并且，数据集中存在噪声会使得漏洞检测模型的评估结果不可信。

3.4.2 去噪方法的有效性

在本实验中，我们分别在合成数据集 SARD 和真实数据集 FFMpeg+Qemu 上比较了 CL 和 DT 两种去噪方法的有效性。对于 SARD，我们选择它是因为它具有高质量的标签，因此我们可以将其视为一个完全干净的数据集，并建立噪声可

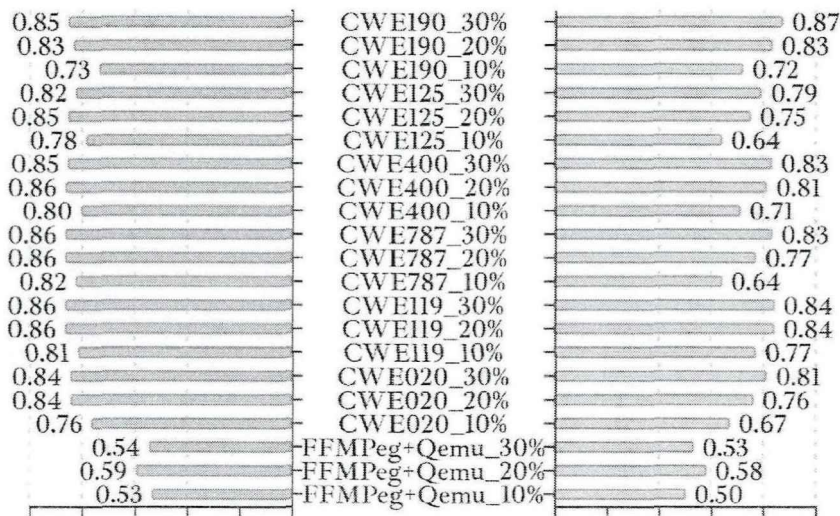


图 3-5 DeepWukong 结合 CL 和 DT 分别在 SARD 和 FFMpeg+Qemu 上 F1-Score_n 的

控制的实验。对于 FFMpeg+Qemu, 我们选择只修改一个文件的 commit, 以使其尽可能干净。总体而言, 我们通过翻转标签在 SARD 和 FFMpeg+Qemu 上设置了 10%、20% 和 30% 的三个噪声级别。对于漏洞检测模型, 我们选择 DeepWukong, 因为它在三个漏洞检测模型中具有最强的漏洞检测能力。

结果: 如图 3-5 所示, 在合成数据集 SARD 上, CL 和 DT 都可以正确地找到所有六种漏洞类型上的大多数噪声样本, 因为在合成数据集上的 F1-Score_n 远远超过了真实数据集。除 CWE190 外, CL 在其他五种漏洞类型中的性能都比 DT 好得多。尤其是 CWE787_10%, CL 的 F1-Score_n 比 DT 高 17.5%。在 CWE190 上, CL 的性能与 DT 相似, 甚至在 20% 和 30% 的噪声水平下 CL 的性能不如 DT。在真实世界数据集 FFMpeg+Qemu 上, CL 和 DT 的 F1-Score_n 都不如在合成数据集上的表现。最高的 F1-Score_n 0.59 也不如在合成数据集上最低的 F1-Score_n 0.64 高。最低的 F1-Score_n 仅有 0.5, 这个表现近乎随机结果。

分析: 降噪技术与漏洞检测器相结合, 揭示了降噪方法的有效性。CL 和 DT 都具有强大的定位和去除噪声标签的能力。当数据规模较大时, CL 表现出比 DT 更好的性能。原因可能与不同去噪方法的复杂结构有关。CL 在交叉验证过程中标记有噪声的样本。用于训练的真实数据集是整个数据集的一部分, 因为交叉验证划分了数据集。CL 进一步减少了用于训练的数据, 并且模型没有被充分学习, 因此当数据集的大小很小时, 例如 CWE190, CL 的性能会下降。去噪后, 噪声比显著降低, 使用 CL 去噪后的数据集变得更干净, 这意味着 CL 在减少噪声标签方面平均比 DT 更有效。

此外, 去噪方法在 SARD 中的有效性更为明显。当应用于真实世界数据集 FFMpeg+Qemu 时, 去噪方法几乎无效。由于 DeepWukong 在 FFMpeg+Qemu 上的拟合能力较差, 与 SARD 上超过 90% 的 F1 分数相比, DeepWukong 只能达到 FFMpeg+Qemu 的 F1 分数的 60%。因此, DT 和 CL 都取决于漏洞检测模型对漏洞数据集的拟合能力。漏洞检测模型的置信水平越高, 去噪方法就越有效, 这将在接下来的实验中得到证明。

结论: 当漏洞检测模型与漏洞数据集非常契合时, 去噪方法在检测和去除数据集噪声标签方面显示出巨大的潜力。在我们的实验中, CL 通常比 DT 提供更好的性能和稳定性。

3.4.3 漏洞检测模型对去噪方法有效性的影响

在本实验中, 我们探讨了漏洞检测器性能与去噪效果之间的相关性。我们分别比较了 DeepWukong、SySeVr 和 VulDeePecker 使用 CL 去噪的性能。对于漏洞检测, DeepWukong、SySeVr 和 VulDeePecker 具有相同的检测粒度, 它们都以语句级粒度为目标。关于检测漏洞的能力, DeepWukong>SySeVr>VulDeePecker,

因此非常适合我们的实验目的。我们只评估 SARD，因为去噪方法的有效性在真实数据集上并不显著。

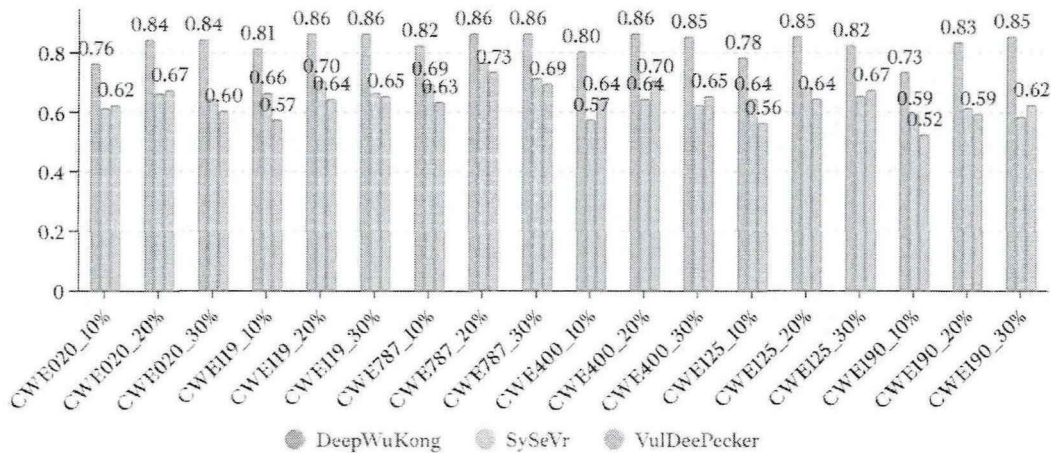


图 3-6 对比 DeepWukong, SySeVr 和 VulDeePecker 分别使用 CL 去噪的 F1-Score_n

结果：图 3-6 显示了使用 CL 的三个漏洞检测模型在不同漏洞类型和不同噪声级别上的性能。可以看出，DeepWukong 可以在所有漏洞类型和噪声等级上达到最高的 F1-Score_n。即使训练集的 30%样本是噪声数据，DeepWukong 也能有高于 0.8 的 F1-Score_n。在 CWE119_30%上 DeepWukong 的 F1-Score_n 最高，且达到 0.86，分别比 SySeVr 和 VulDeePecker 高 0.2 和 0.21。然而当训练集中仅包含 10%噪声时，DeepWukong 的 F1-Socre_n 偏低。对比 SySeVr 和 VulDeePecker 的 F1-Score_n,在大多数数据集（如 CWE119、CWE787、CWE125 和 CWE190）上，SySeVr 的性能高于 VulDeePecker。令人惊讶的是，在 CWE400 上，VulDeePecker 远高于 SySeVr。尤其 CWE400_10%时，VulDeePecker 比 SySeVr 高 0.07。

分析：我们观察到，去噪方法 CL 非常依赖于所采用的漏洞检测模型的性能。漏洞检测模型检测漏洞的能力越强，使用 CL 的去噪效果越高。考虑到 DeepWukong 的漏洞检测能力在三个漏洞检测模型中最强，DeepWukong 使用 CL 去噪的结果表现出最佳性能。当真实噪声比小时，DeepWukong 的 F1-Score_n 更低，因为此时，CL 具有更高的误报率，这将使得更多的非噪声样本标记被误报为噪声样本。理论上，SySeVr 检测漏洞的能力高于 VulDeePecker。因此，当使用 CL 去噪时 SySeVr 的 F1-Score_n 也应高于 VulDeePecker。除了两种漏洞类型 CWE020 和 CWE400 之外，SySeVr 和 VulDeePecker 的性能基本符合预期。这是因为与 SySeVr 相比，VulDeePecker 更适合 CWE020 和 CWE787，因为这两种漏

洞类型是由数据依赖性造成的，VulDeePecker 更关注与漏洞触发行相关的数据流。因此，漏洞检测模型对漏洞数据集的检测能力越高，去噪方法的有效性就越好。

结论：结果表明，当漏洞检测模型具有更强的检测能力时，去噪方法产生了更好的效果。

3.4.4 通过去噪增强漏洞检测模型

在本实验中，我们探讨了在去噪后的数据集上训练漏洞检测模型是否可以提高模型在无噪声测试集上的性能。我们从相应的噪声数据集中去除 CL 发现的噪声样本，用去噪后数据集重新训练 DeepWukong，并用无噪声测试集进行评估。为了验证我们所提出的去噪方法——基于集成学习和置信剪枝的漏洞数据集去噪的有效性，我们增加了一组 CL+Ensemble 与 CL+DeepWukong 的对比实验，来验证我们提出方法的有效性。

结果：图 3-7 所示，在所有漏洞类型上，DeepWukong 在去噪后的数据集上比去噪前表现更好。然而，去噪后的改善并不明显，在 F1-Socre 上最高增幅仅为 8.97%，如图 3-7 的 CWE125_30%列所示。当数据集中有 30%的噪声时，去噪后 DeepWukong 的性能显著提高，当数据集中有 10%的噪声时，去噪后 DeepWukong 的 F1-Score 仅提高约 2%。如图 3-8 所示，当我们将使用 CL 结合集成学习进行去噪之后，与 CL+DeepWukong 相比，DeepWukong 全部数据集上都有了普遍的提升。尤其是在 CWE190_10%上，竟然提升了 5.88%的 F1-Score。

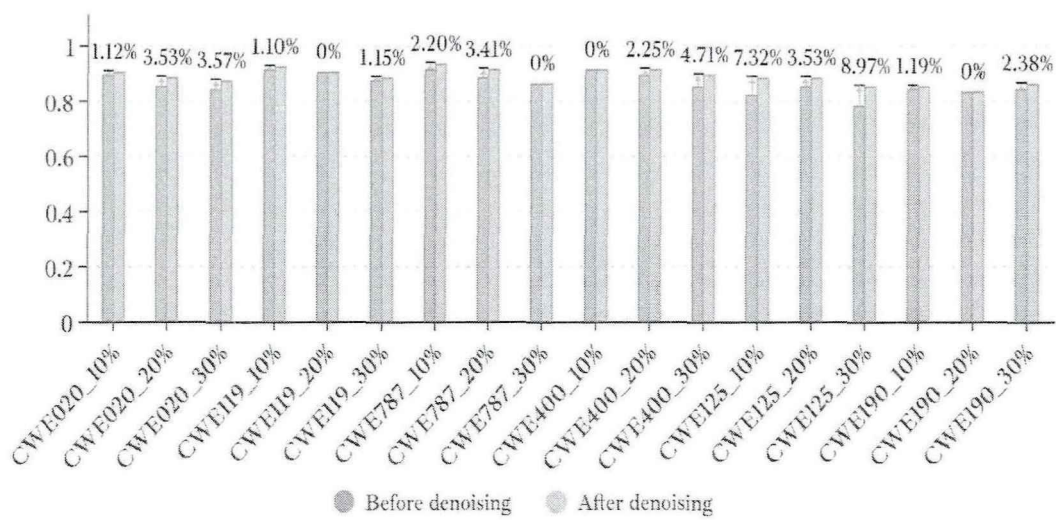


图 3-7 DeepWukong 在 CL 去噪后与去噪前的 F1-Score 对比

分析：从我们的统计数据可以得出结论，当我们在去除 CL 给出的噪声样本后重新训练 DeepWukong 时 DeepWukong 的性能仅略有改善。原因是 CL 在发现有噪声的样本时会有更多的误报。当我们去除真实的噪声样本和假阳性时，一些非噪声样本也会被同时去除。与原始数据集相比，用于训练的数据量大大减少，这导致模型训练不足。因此，我们采用集成学习的方式来融合不同模型结构的决策边界，从而降低 CL 的误报率，也就减少了 CL 将非噪声样本识别为噪声样本的数量，从而有更多可用于模型学习的正确样本，对漏洞检测模型性能提升有极大的帮助。

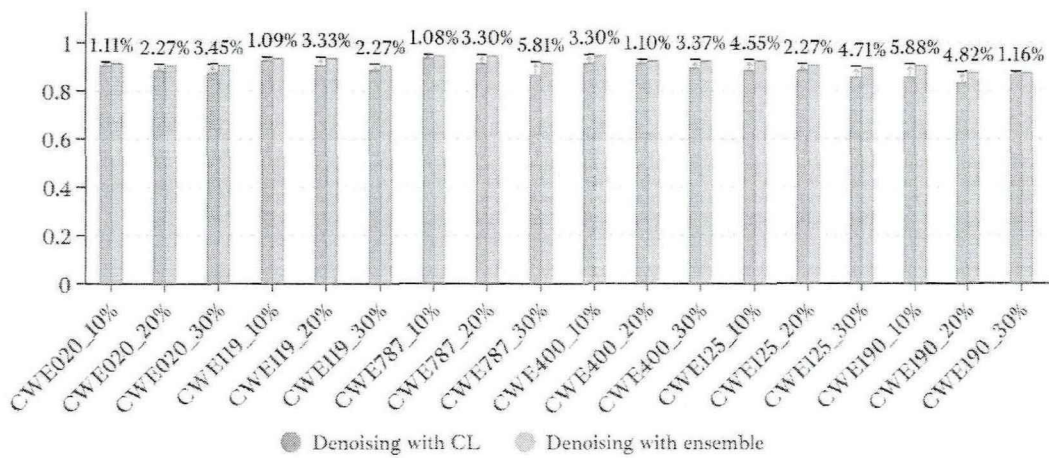


图 3-8 CL+DeepWukong 与 CL+Ensemble 去噪后的 F1-Score 对比

结论：基于集成学习和置信剪枝的去噪方法能够有效的提升漏洞检测模型的性能。

3.5 本章小结

在本章节中，我们描述了最先进的漏洞检测方法通常用于训练模型的数据集包含大量噪声标签的现象，并发现了如何提高数据集标签质量的关键问题。我们提出了一种解决方案，通过去除漏洞数据集中的噪声标签来提高漏洞数据集的标签质量。我们介绍了 CL 和 DT 两种去噪方法，并提出了一种基于集成学习和置信剪枝的去噪方法。我们证明，去噪方法非常依赖于漏洞检测模型对漏洞数据集的拟合能力，并且去噪方法在提高漏洞检测性能方面具有巨大潜力。此外，我们还揭示了数据集中存在噪声对漏洞检测模型的影响，以及三个漏洞检测模型 DeepWukong、SySeVr 和 VulDeePecker 从噪声数据中学习的能力。我们通过改进漏洞数据集的标签质量，使漏洞检测模型更容易学习真实的漏洞特征，为提升基于学习的源代码漏洞检测模型的鲁棒性提供了可行的路径。

第四章 基于语义保护代码变换的数据增强

4.1 问题描述

我们在第三章中已经详细探讨了前沿的基于深度学习的漏洞检测方法所使用的数据集存在的问题，除了我们已经指出的噪声标签普遍存在的问题会导致用这些数据集训练过的模型鲁棒性下降之外。我们还发现了数据集中其他影响模型鲁棒性的问题。如表 3-1 所示，最常被使用的数据集分两种类型。

第一种以 SARD 为代表的合成数据集，此类数据集是根据安全专家预定义的漏洞缺陷模式，自动的生成一些用于研究学习安全漏洞的代码示例。此类漏洞数据集数据量较大且标签可靠，可以满足一些常用模型的训练需求，但是它存在一个极大的缺点，就是漏洞模式太过于单一。因为合成数据集模式单一的特点，导致领域内的工作都喜欢在 SARD 上进行测试评估。这是因为，单一的漏洞模式容易被神经网络记忆学习，有漏洞和无漏洞的样本仅通过单一的漏洞模式就可以容易区分，没有太多多余的无关代码进行干扰，因此在最后的测试评估中能够达到非常高的评估数值。但是，这样就会导致一个严重的问题，我们无法得知模型是根据代码的形式化差异对有漏洞的样本和无漏洞的样本进行区分，还是真正学习到了漏洞触发的规律。这也就导致很多工作其实是徒劳的，看似在评估时得到了很高的分数，其实对真实世界的漏洞检测没有太大的贡献。

第二种类型以 Qemu+FFMPeg 为代表的真实世界数据集，此类数据集是通过公开漏洞数据库发布的漏洞记录来进行获取的，每个漏洞都会有唯一的漏洞 ID，且与存在漏洞的软件版本一一对应。通过爬取此类公开网站，再使用一些手段筛选出与漏洞相关的软件版本，将对应的修复前后的软件版本代码分别下载，作为有漏洞和无漏洞的代码样本。此类数据集是来源于现实世界的代码，因此其中的漏洞也是真实代码中的漏洞，模式极其复杂，其中存在许多冗余代码对模型学习产生干扰。因为现实世界软件中的漏洞总是稀有的且是危险的，很多企业不愿进行公开，一经发现马上修复。因此很难收集到大量的漏洞样本，仅凭公开的漏洞，是无法获得充足的样本的，又因公开漏洞数据库的漏洞收录跨度很大，包含不同年份及不同工程之间的跨度，更加使得模型不易学习。此外，漏洞样本数量不足，模式单一会导致模型过拟合，从而受到对抗攻击。

综合以上两点问题，第一点漏洞模式简单且单一，第二点漏洞样本数据量少，我们提出通过基于语义保护的代码变换对原始数据集进行数据增强。我们仅对源代码的形式进行修改，不对代码的功能和语义进行修改，保证同样的输入在代码

变换前后保持一致的输出。经过数据增强，会丰富数据集中样本的漏洞模式，对提升模型的鲁棒性有极大的帮助。

4.2 代码变换算法原理

在本小节中，我们具体介绍对漏洞数据集中的源代码做语义保护的等价转换，它仅仅修改代码的语法，但保持代码语义不变。因此，我们提出的方法中的变换与变异测试中的变异算子不同。变异测试修改源代码，用于在程序中插入 bug，以衡量测试套件的有效性。因此，变异测试中的变异算子通常在生成新代码时更改代码语义。然而，我们的目标是实现语义保护的等价代码转换。为了实现这一目标，我们总共支持 15 个唯一的代码变换算子，包括变量重命名、使用等效语义更改语法结构、添加垃圾代码、删除无关代码、重新排序独立代码等变换方式，具体描述如代码变换规则 4.2.1 节所示。

4.2.1 代码变换规则

表 4-1 代码变换规则

变换操作	描述	原始代码	改变代码
操作 1	修改函数名和变量名	int i;	int i1;
操作 2	For 循环改成 while 循环	for(i=0;i<10;i++){ BodyA }	i=0; while(i<10){ BodyA i++; }
操作 3	While 循环改成 for 循环	while(i<10){ BodyA }	for(;i<10;){ BodyA }
操作 4	Do 循环改成 while 循环	do{ BodyA }while(i<10);	BodyA while(i<10){ BodyA }

表 4-2 代码变换规则（续上表）

变换操作	描述	原始代码	改变代码
操作 5	If elseif 改成 if else	if(grad<60) BodyA else if(grad<80) BodyB else BodyC	if(grad<60) BodyA else{ if(grad<80) BodyB else BodyC }
操作 6	If else 改成 if elseif	if(grad<60) BodyA else{ if(grad<80) BodyB else BodyC }	if(grad<60) BodyA else if(grad<80) BodyB else BodyC
操作 7	Switch 改成 if elseif	switch(a){ case 60: BodyA case 70: BodyB default: BodyC }	if(a==60) BodyA else i f(a==70) BodyB else BodyC
操作 8	改变关系表达式	a<b	b>a
操作 9	修改一元运算	i++;	i=i+1;
操作 10	修改为自增运算	i+=1;	i=i+1;
操作 11	修改常量	8	(a-b)//8=a-b
操作 12	修改变量定义	int b=0;	int b;b=0;
操作 13	增加垃圾代码	if(a){ BodyA }	if(a) { BodyA if(0) return 0; }
操作 14	改变没有依赖的语句顺序	a=b+10; c=d+10;	c=d+10; a=b+10;
操作 15	删除 print 语句和注释	printf("test"); //comments	printf("test"); //comments

如表 4-1 所示, 我们共采用了 15 种语义保护的代码变换, 第一列为操作序号, 第二列简要描述了每个操作如何生效, 第三列显示了等价变换之前的简单代码示例, 最后一列给出了转换后的代码。每种操作都代表着一种语义保护的代码变换。

操作 1 会对代码片段中的变量名和函数名进行变换, 是使用最普遍且相对简单的一种代码变换方式。

操作 2 与操作 3 是互逆的一种操作, 分别是将 For 循环的代码结构转换为 While 循环以及将 While 循环的代码结构转变为 For 循环。

操作 5 将 Do While 循环结构转换为纯 While 循环结构, 因为 Do While 循环会先执行一次循环体, 在进行条件判断, 因此在转换之后需要在条件判断之前另外执行一次循环内容。

操作 5 将 If elseif 结构转换为 If else 结构。

操作 6 将 If else 结构转换为 If elseif。

操作 7 将 Switch 结构转换为完全的 If else 结构。

操作 8 改变了关系表达式, 将表达式转换为他的非逆。

操作 9 和操作 10 为互逆操作, 分别是将自增操作转换为一元运算以及将一元运算转换为自增操作。

操作 11 将常量数值转化为表达式计算形式, 此转换方式会随机生成一个表达式, 表达式计算结果为原本的常量数值。

操作 12 修改变量定义的规则, 将变量的赋值与定义分别开来。

操作 13 为原始的代码片段中添加永远不会执行的垃圾代码。不会对程序语义产生任何影响。

操作 14 对原始代码片段中互相没有数据依赖的语句进行的顺序替换。

操作 15 删除了原始代码片段中对程序语义不会产生影响注释语句和打印日志语句。

如图 4-1 我们选择了 15 种操作中的 3 种作为代表, 来展示源代码片段经过语义保护的代码变换会产生怎样的改变。

经过操作 1 变换后的源代码, 原始的变量名由 hp 改变为 h7sMlt, 原始的变量名 addr 改变为 ll9bnru0。

经过操作 11 变换后的源代码, 常量 64 改变为 455-391。

经过操作 13 变换后的源代码，增加了永远不会执行的代码块，因为 if 条件永假，所以永远不会执行。

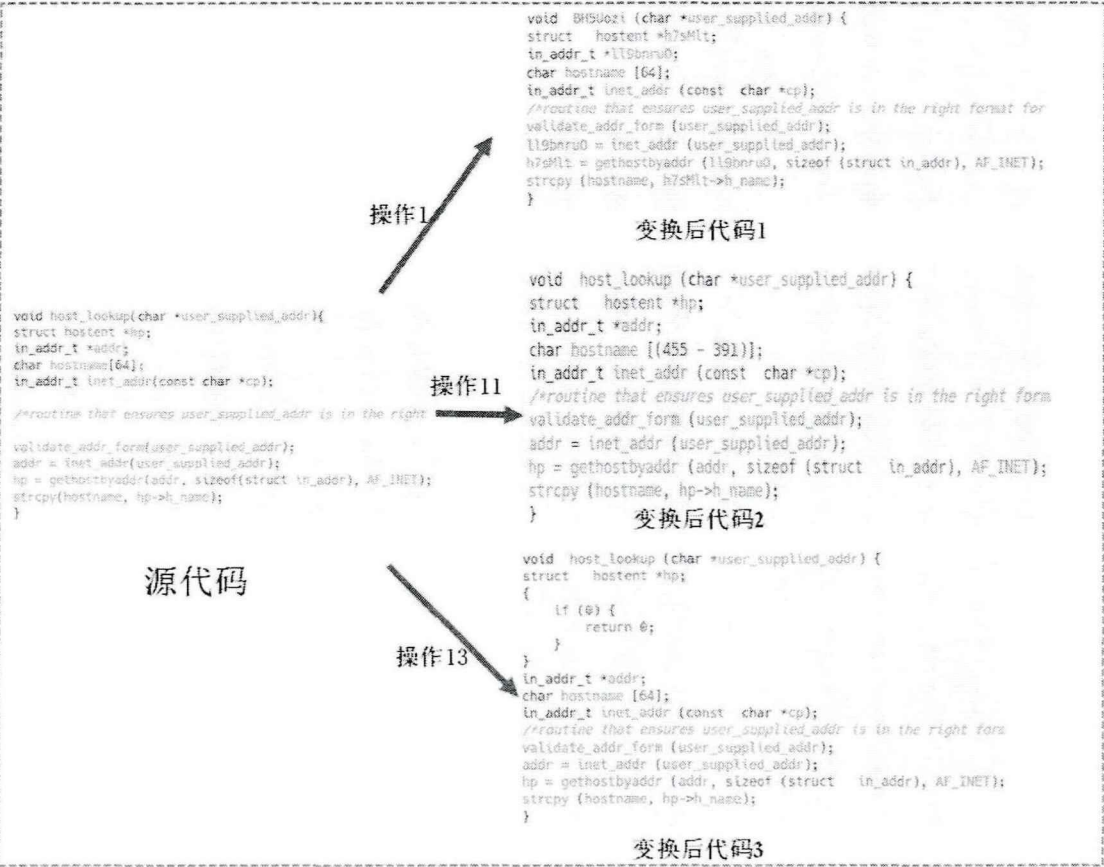


图 4-1 语义保护的代码变换实例

4.2.2 代码变换策略

因为 TXL 是针对语法结构的程序语义修改，如果我们想要对一段代码生成一个经过等价变换的副本，首先我们需要从 15 种代码变换中选择一个代码变换规则，尽可能保证每个规则被公平选取。其次，我们如果想要将一段目标程序中的 For 循环修改成 While 循环，我们需要从目标程序中的多个 For 循环结构中选择一个或多个 For 循环进行修改，而不是全部修改。如果全部修改的话，达不到我们要的多样性的目的。我们共采用了如下两个代码变换策略。

随机选择变换规则：第一个策略，我们采用最简单的随机策略，保证每个规则都被公平使用，保证每个语法结构都能够得到变换，首先我们对输入程序进行一遍扫描，记录每个语法结构的序号以及出现的次数，然后我们在出现过的语法结构中随机选取一个语法结构。在选取语法结构之后，我们从该语法结构的候选序号中随机选取一个序号进行相应的等价变换，至此我们完成了一次代码的等价

变换。简单来说，就是对于一段代码，我们只随机选择一种变换规则，对一处语法结构进行变换。此种策略能够达到最小幅度的数据增强。

选择全部变换规则：第二个策略，我们在第一个策略的基础上，在选择代码变换规则时，将所有能够使用的规则都使用一遍。此种策略能够生成的样本数量比第一种策略多数倍。

4.2.3 代码变换实现细节

4.2.3.1 TXL

表 4-1 中所有的语义保护的代码变换操作都是基于 TXL 实现的。TXL 是一种独特的编程语言，专门用于设计支持计算机软件分析和源代码转换任务。这是十五年来集中研究基于规则的结构转换作为快速解决复杂计算问题的范例的不断发展的结果。TXL 编程语言是一种基于规则的语言，具有统一性、隐含迭代和深度模式匹配。每个 TXL 程序有两个组件：一个要转换的结构描述，指定为直接解释的 BNF 语法，上下文无关的歧义形式。一组结构转换规则，指定为模式/替换对，使用函数编程组合。TXL 的独特之处在于，它有一个纯功能的上层结

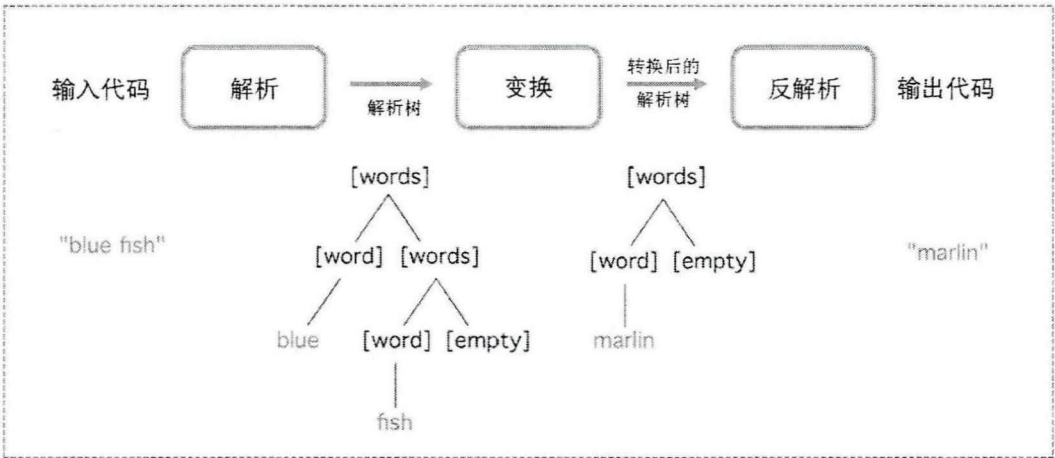


图 4-2 TXL 原理

构，提供范围、抽象、参数化和递归，而不像 Prolog 那样的结构重写规则，提供模式搜索、统一和隐式迭代。TXL 的形式语义和实现基于形式树重写，但由于规则规范的示例样式，树对用户来说很大程度上是隐藏的。TXL 最擅长涉及结构分析和形式符号转换的任务，如编程语言、规范语言、结构化文档等。

如图 4-2，每个 TXL 程序分三个阶段运行。第一个阶段是解析阶段。解析器接受整个输入代码，对其进行标记，然后根据 TXL 程序的语法定义对其进行解析，以生成解析树。TXL 程序的第二个阶段是完成所有转换工作的阶段。它获

取输入的解析树，并将其根据定义好的转换规则转换为对应于所需输出的新树。最后一个阶段只是对转换器生成的树进行反解析，生成输出代码即转换后的代码。

我们采用的转换规则都是轻量级的，仅用到了 TXL 的词法分析器在语法层面进行转换，并没有对代码的控制流图，程序依赖图或调用图进行修改。

4.2.3.2 代码变换流程

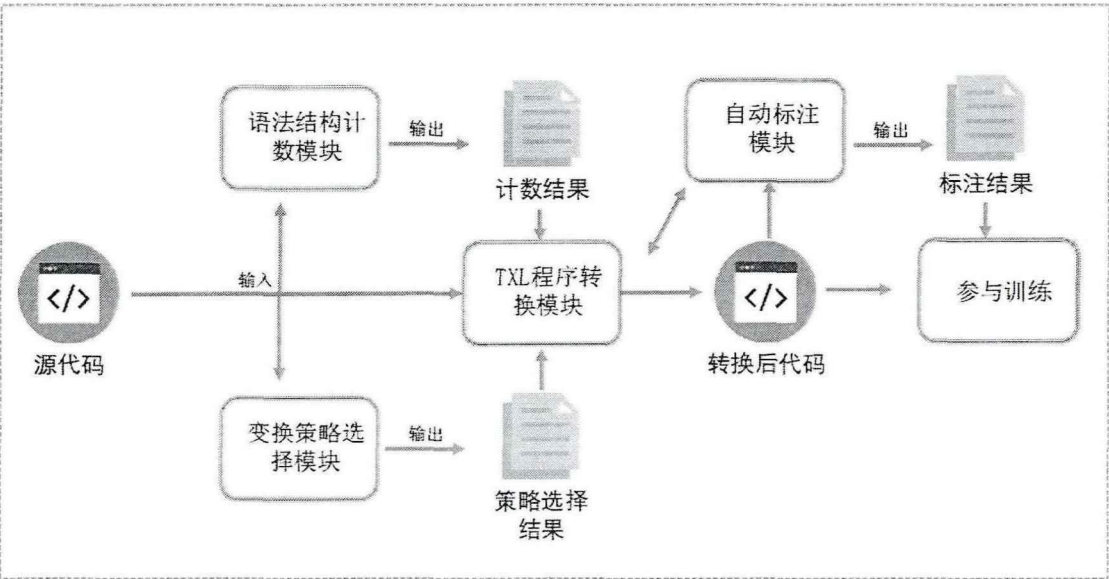


图 4-3 代码变换流程图

如图 4-3，我们详细的介绍了源代码从生成新代码到新代码参与训练所需要的全部技术细节。

首先待变换的源代码输入系统，我们需要通过语法结构计数模块来检测待变换的代码片段中包含的每一种语法结构的数量以及每一个语法结构所在的序号，这一过程通过 TXL 来实现，输出结果为每一种语法结构生成一个 .count 后缀的文件记录该段代码包含该语法结构多少个。

然后，根据输入参数的设置，我们可以选择随机变换策略以及全部变换策略来进行下一步变换。输出一个名为 action 的文件来决定该次变换所选择的操作。

在得到策略选择方案以及程序语法结构计算结果之后，我们通过 TXL 程序转换模块来生成新的代码片段，因为我们生成的新代码是用于训练的所以对于那些原始包含漏洞行的代码，我们需要在它变换之后也能找到对应的漏洞行，因此在参与变换之前，我们在漏洞行之前插入一个标志性的语句。

在执行 TXL 程序转换之后，我们通过自动标注模块来找到新生成的代码片段中漏洞行所在的位置，并记录标注结果。

在得到标注结果与新的源代码之后，根据所使用的漏洞检测方法进行相应的预处理过程，就可以参与进一步的数据增强训练。

4.2.4 代码变换优化方案

由于我们在前面提出的代码变换策略是随机的进行一些代码结构上的变换，目的是为了增加样本的多样性，但是盲目的去增加样本数量是否能够对模型训练带来稳定的帮助暂时还不得而知，并且，有些代码变换之后的样本还会对模型学习漏洞特征带来一些不可避免的干扰。因此，我们希望能够通过解释模型的预测结果来对当前样本做一些有针对性的代码变换，以帮助模型有目的得学习一些与漏洞相关的特征。通过这种方式，只需进行少量的代码变换，就能够达到随机代码变换相同的效果。

算法 1 有针对性的代码变换训练算法

算法 1：有针对性的代码变换训练过程	
输入：漏洞检测模型 model；数据集 D；迭代次数 n；预热步骤 w	
输出：训练完成的漏洞检测模型 model	
1	模型 model 初始化
2	for i in n do
3	if i <= w then
4	model = train(model, D)
5	else
6	model = train(model, D)
7	初始化 D1 为空
8	for (x,y) in D then
9	if y == 1 then
10	line_no = get_vul_line(x)
11	explain_res = explain(model, x)
12	if line_no not in explain_res then
13	x_new = code_trans(x,line_no)
14	else
15	x_new = x
16	else
17	x_new = random_code_trans(x)
18	D1 = D1 x
19	D = D1

随机代码变换需要在训练前就生成所有的新样本，将新样本与旧样本混合参与模型训练。与随机代码变换不同之处在于，有针对性的代码变换需要可解释方

法对模型预测结果的解释来辅助完成，并且代码变换是边训练边执行的，可解释性方法将在第五章中详细介绍。

有针对性的代码变换整体过程如

算法 1 所示，首先初始化漏洞检测模型 `model`，然后迭代训练 `n` 次，前 `w` 次为预热过程，目的是让模型对原始数据集有一个初步的了解。当迭代次数大于 `w` 之后，对于当前数据集 `D` 中每个漏洞样本（标签为 1）使用 `model` 进行预测并使用解释方法 `explain` 进行解释，解释结果会反馈当前样本中影响模型预测结果最重要的代码行，结合漏洞行，我们可以知道影响模型决策最重要的几行中是否包含漏洞行，若包含，我们认为当前模型学习到了漏洞的特征，若不包含，我们认为

算法 1：有针对性的代码变换训练过程

输入：漏洞检测模型 `model`；数据集 `D`；迭代次数 `n`；预热步骤 `w`

输出：训练完成的漏洞检测模型 `model`

```

1  模型 model 初始化
2  for i in n do
3      if i  $\leq$  w then
4          model = train(model, D)
5      else
6          model = train(model, D)
7          初始化 D1 为空
8          for (x,y) in D then
9              if y == 1 then
10                  line_no = get_vul_line(x)
11                  explain_res = explain(model, x)
12                  if line_no not in explain_res then
13                      x_new = code_trans(x,line_no)
14                  else
15                      x_new = x
16              else
17                  x_new = random_code_trans(x)
18              D1 = D1 | x
19          D = D1

```

对于该样本漏洞没有学习到它的特征，因此我们采用语义保护的代码变换策略对该行代码进行随机变换。目的是辅助模型学习与漏洞行相关的代码特征。对于非漏洞样本我们采用随机代码变换以增加数据集的多样性，将此次迭代生成的新数

据集 D1 作为下一次迭代的数据集 D，当全部迭代完成之后，输出训练好的模型 model。

4.3 实验设置

本章我们要通过实验来探究基于语义保护代码变换的数据增强能否提升模型在检测漏洞上的性能，实验目的与第三章异曲同工。因此，除了在漏洞检测模型 BaseLine 上的选择与第三章不同，并且本章不需要对数据集设置人工可控比例的噪声样本，其余实验配置均未改变。在 BaseLine 的选择上，因为数据增强之后的数据量较大，且不同漏洞检测方法对数据进行预处理的手段不同，评估多个模型需要重复多次预处理步骤，需要花费大量的服务器运行时间。因此，本实验仅选择最复杂的模型结构 DeepWukong 来评估数据增强的有效性。对于可解释性方法，由于 DeepWukong 是基于 GNN 的漏洞检测模型，因此我们使用经典的 GNNExplainer^[45]作为可解释性方法。

4.4 实验及结论

为了评估基于语义保护的代码变换数据增强的有效性，我们使用 DeepWukong 在 SARD 和 Qemu+FFMPeg 上进行了三组实验。总体而言，我们的评估针对以下方面：

- 1) 基于语义保护代码变换的数据增强对漏洞检测模型的影响。
- 2) 不同程度的数据增强对漏洞检测模型的影响。
- 3) 有针对性的语义保护代码变换训练方案对漏洞检测模型的影响。

4.4.1 数据增强的有效性

本小节的目的是探究基于语义保护代码变换的数据增强是否能够有效的提升源代码漏洞检测模型的检测能力。我们使用拥有最复杂模型结构的 DeepWukong 作为漏洞检测 BaseLine 方法，并使用 SARD 和 Qemu+FFMPeg 作为漏洞数据集 BaseLine。我们通过记录 DeepWukong 分别在 SARD 和 Qemu+FFMPeg 上在数据增强前后的 F1-Score 的变化来评估数据增强的有效性。为了能够更加有力的体现出数据增强的作用，我们控制了最小程度的数据增强，来降低样本量大幅度增加带来的影响。因此我们选择了第一种代码变换策略来进

行实验。

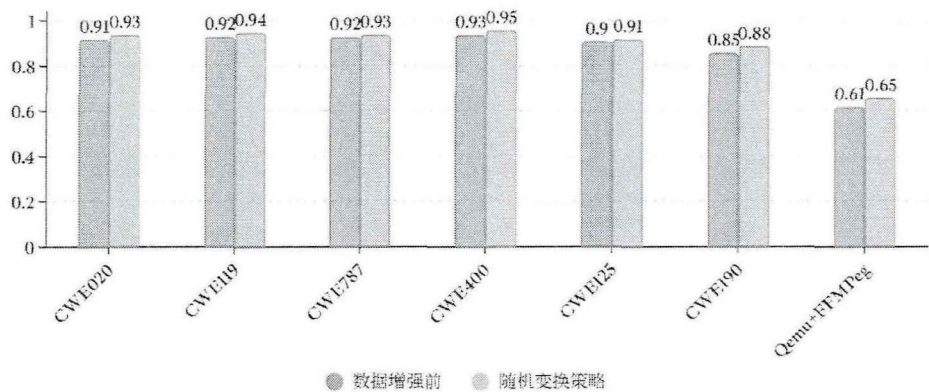


图 4-4 DeepWukong 数据增强前后的 F1-Socre 对比

结果：如图 4-4 所示，我们通过数据增强扩充了一倍的数据，重新训练 DeepWukong 后无论是在 SARD 上还是在真实世界数据 Qemu+FFMPeg 上，DeepWukong 的 F1-Score 都有不同程度的提升，这充分说明数据增强对模型起到了一定的帮助作用。但总体来说，提升不算明显，在 SARD 上最高提升了 3 个百分点。令人吃惊的是在 Qemu+FFMPeg 上居然提升了 4 个百分点。

分析：F1-Score 普遍的提升是因为语义保护的代码变换能够给模型展现不同形式的漏洞样本，让漏洞检测模型更加关注漏洞语义的特征忽略那些形式上的语法特征。在 SARD 上提升不明显的原因是，SARD 数据集结构较为简单，模型已经能够很容易区分了，提升空间较小。在 Qemu+FFMPeg 上的提升，说明语义保护的代码变换，对于模型学习复杂逻辑的漏洞代码也有一定的帮助作用。

结论：通过基于语义保护的代码变换方法对漏洞数据集的漏洞代码模式进行多样化能够提升漏洞检测模型的性能。

4.4.2 不同程度数据增强的影响

在确认基于语义保护代码变换的数据增强能够对模型性能有所提升之后，我们更加关注数据增强的程度不同是否会对模型有不同程度的提升。因此本次实验的目的是分别使用两种语义保护的代码变换策略，构建不同数量的样本，来进行数据增强。本实验中，我们评估了 DeepWukong 在 SARD 的六种漏洞类型和 Qemu+FFMPeg 上，在未经数据增强时、最小程度数据增强时以及最大程度数据增强时的漏洞检测性能（F1-Score）。

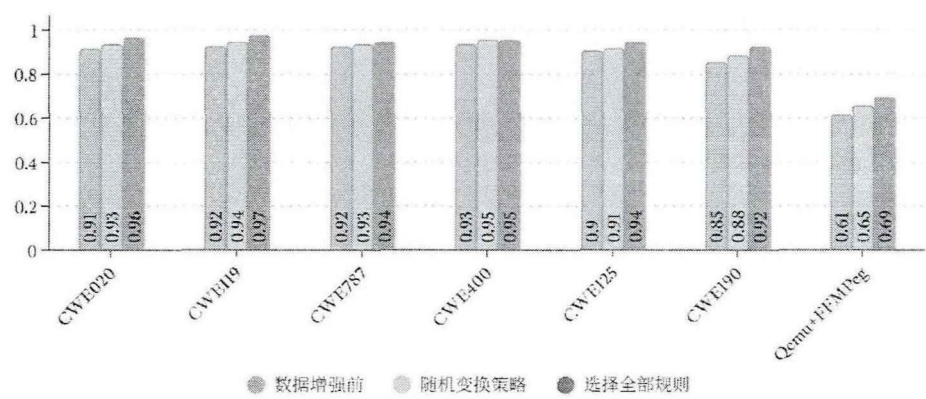


图 4-5 DeepWukong 使用两种数据增强策略的 F1-Score 对比

结果：我们使用第一种语义保护的代码变换策略扩充了 1 倍的数据量，使用第二种变换策略扩充了约 7 倍的数据量，如图 4-5 所示，DeepWukong 的 F1-Score 相较于数据增强前有大幅度的提升，最高提升了 8 个百分点，这充分说明了数据量大幅度增加以后，模型能够从多样化的样本中学习到更准确的漏洞特征。训练模型的数据量越多，模型检测漏洞的能力也越强。但是，值得注意的是，F1-Score 的提升并非与数据量的提升成正比，提升数据量对模型性能的提升会随着数据量增加而减小，这可以从随机变换策略相较于数据增强前的提升比例以及选择全部规则相较于随机变换策略的提升比例中明显看出。

分析：在提升 7 倍训练样本数量之后，训练集中包含各种经代码变换后的样本，同一样本可能扩充至多 15 个不同语法相同语义的样本，这些样本的拥有相同的漏洞逻辑，在训练模型时能够帮助漏洞检测模型排除一些形式化特征，提升模型的泛化能力，从而能够泛化更多的未见过的漏洞样本。但是数据量并不与模型提升成正比，因为初期的数据增强确实能够提升模型泛化能力，但是对于固定代码模式的数据，进行太多倍的数据增强反而会导致模型对此种模式过拟合。

结论：大幅度的数据增强确实能够提升模型的漏洞检测能力，但是数据增强的程度越大，对模型提升的幅度越小。

4.4.3 有针对性的语义保护代码变换面对随机代码变换是否有优势

我们在 4.4.1 中已经通过实验证明了通过随机代码变换是扩充一倍的数据是能够有效的提升漏洞检测能力的，但是我们认为随机代码变换存在一些弊端，随机生成的新样本可能会对漏洞检测模型产生误导，并且我们通过 4.4.2 证明了一味的增加数据量也不会稳定的带来提升，因此我们寻求一种高效的代码变换方式，并通过此方法来帮助模型学习漏洞的特征。本实验中，我们评估了 DeepWukong 在 SARD 的六种漏洞类型和 Qemu+FFMPeg 上，在未经数据增强时、最小程度数据增强时以及有针对性的数据增强的漏洞检测性能（F1-Score）。

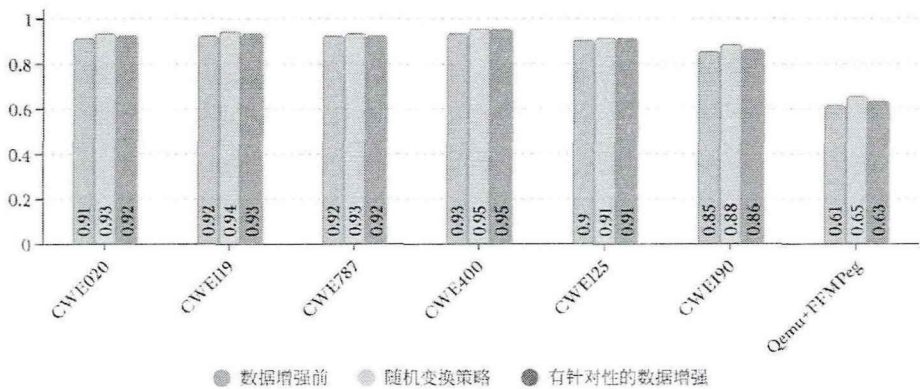


图 4-6 DeepWukong 随机数据增强以及有针对性数据增强的 F1 对比

结果：我们使用最小数据增强的代码变换策略扩充了 1 倍的数据量，使用有针对性的数据增强策略边训练边进行数据增强，如图 4-6 所示，当时使用有针对性的数据增强策略时，DeepWukong 的 F1-Score 相较于数据增强前的提升，在多个数据集上都与随机变换策略相同，这说明了有针对性的数据增强对模型漏洞检测性能提升效果普遍与随机代码变换策略相同，但是有针对性的数据增强能够在提升漏洞检测性能的同时依靠可解释模型将模型的解释结果向着漏洞特征靠拢。

分析：随机代码变换的数据增强本质上使用的数据量是有针对性的代码变换方案的 2 倍，因此有针对性的数据增强策略在不扩充数据量的条件下能够达到与随机代码变换相同的增益效果，已经足够体现它的有效性，但是随机代码变化由于其解释阶段需要对每个预测结果进行解释，并且解释过程是一个搜索过程，需要耗费较多时间，因此不适合数据量大的时候使用。

结论：有针对性的数据增强能够在不扩充数据量的情况下达到与随机代码变换使用 2 倍数据的增强效果，但是训练过程中时间上的耗费较大。

4.5 本章小结

在本章节中,我们描述了最先进的漏洞检测方法通常用于训练模型的数据集中漏洞模式单一而且数据量不足的现状,并提出数据增强来增加模型的泛化能力,提升模型的鲁棒性。因为源代码样本的特殊性,我们提出了一种解决方案,通过语义保护的代码变换来对漏洞代码数据集中的代码多样性进行扩充。我们实现了 15 种语义保护的代码变换规则,并提出了 3 种数据增强策略。经过实验,我们证明了通过基于语义保护的代码变换方法对漏洞数据集的漏洞代码模式进行多样化能够提升漏洞检测模型的性能,并且一定程度上的数据增强能够有效的提升漏洞检测模型的性能。由于随机代码变换的数据增强会大量扩充训练数据的样本量,因此我们为了验证在不扩充数据量时代码变换是否有效,提出了有针对性的数据增强,并证明了其能够在数据量不变的前提下通过扩充代码多样性来提升模型能力。我们提出的数据增强方式能够有效的提升漏洞检测模型检测漏洞的能力,为提升模型鲁棒性提供了可行的方案。

第五章 多维度的漏洞检测模型鲁棒性度量方法

5.1 问题描述

我们在第一章中描述了现阶段兴起的一系列基于深度学习的漏洞检测模型普遍存在的一系列问题，我们发现了这些工作中存在的三个比较严重的问题：

(1) 许多基于深度学习的源代码漏洞检测工作泛化能力较差，在将训练好的模型应用到与训练集非同源的测试集上进行评估时，性能下降幅度较大。

(2) 基于深度学习的源代码漏洞检测容易受到对抗样本攻击，当攻击者有目的对输入样本进行设计时，漏洞检测模型容易被绕过检测。

(3) 前沿的基于深度学习的源代码漏洞检测模型所常用的漏洞数据集以及漏洞数据收集方法中，由于一些细节上的不完善导致生成样本标签时引入了噪声标签。噪声标签的引入最终使得模型学习到了一些错误的特征。

在本课题中，我们将以上三个问题归结为基于深度学习的漏洞检测模型鲁棒性不足的问题。由于导致模型鲁棒性不足的原因可能有多个，包括网络模型结构设计、数据集构建以及模型训练方法等方面。首先，不同的漏洞检测方法会使用不同的网络模型结构来拟合漏洞特征，因为漏洞检测领域没有一个像图像分类领域的 ImageNet 一样可以绝对信赖的 BaseLine 数据集，很难去衡量一个模型的结构是否设计的足够鲁棒。其次，对于模型训练过程而言，我们参考的这些基于学习的漏洞检测方法的训练方式基本上都是一致的，将预处理后的样本送入模型经过多轮的前馈计算及梯度反传来拟合一个二分类任务，所以无法区分训练过程对模型鲁棒性的影响。最后，由于深度学习是数据驱使的，数据决定了模型性能的上界，且我们提出的三个问题都可以通过改善数据集来进行弥补。并且，我们已经在第三章和第四章中针对这三个问题提出了针对数据集的解决方案，并得到了一些成效。

由于目前在漏洞检测领域，大部分研究者们都在朝着提升漏洞检测模型的精度这个目标前进。还未曾有人针对模型的鲁棒性进行系统的研究，所以没有一个可用的漏洞检测模型鲁棒性度量方法。针对这个需求以及本章提出的三个问题，本课题提出一个多维度的漏洞检测模型鲁棒性度量方法，来对一些经典的漏洞检

测模型进行鲁棒性度量，并在使用我们提出的鲁棒性优化方法之后，证明其有效性。

5.2 多维度度量指标

本小节根据前期调研结果中能够反映漏洞检测性能及鲁棒性的几个角度来详细介绍我们所设计的多维度漏洞检测模型鲁棒性评估框架。

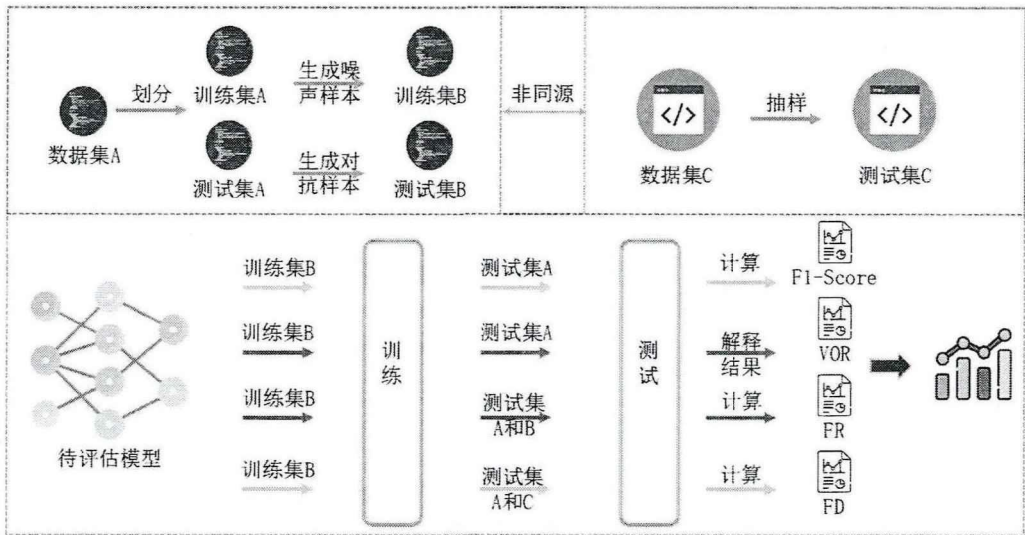


图 5-1 多维度漏洞检测模型鲁棒性度量方法

如图 5-1，我们的度量方法使用了两个非同源数据集 A 和数据集 C 通过四条路径分别从四个维度对待评估漏洞检测模型的综合鲁棒性进行评估。首先我们提供非同源数据集 A 和数据集 C，数据集 A 根据 8：2 的比例划分为训练集 A 和测试集 A，训练集 A 和测试集 A 都是无噪声的干净数据集。在得到训练集 A 之后，我们将训练集 A 中 30% 的样本进行翻转以生成噪声样本，得到训练集 B（带噪声样本的训练集）。随后我们为测试集 A 中的每个样本，使用语义保护的代码变换规则，随机生成一个对抗样本，得到测试集 B（对抗样本集合）。对于数据集 C，我们从中抽样于测试集 A 和 B 相同数量的样本，作为测试集 C（非同源测试集）。

在生成评估漏洞检测模型鲁棒性所需要的全部数据集之后，我们详细解释每个维度所侧重的点，以及评估方式。

F1-Score: 我们认为要评估一个漏洞检测模型是否鲁棒的前提条件是这个漏洞检测模型能够有效的检测漏洞，因为对于一个几乎无法正确识别漏洞样本的漏洞检测模型，评估它的鲁棒性是没有必要的。在这个评估维度，我们更侧重于评

估模型检测漏洞的能力，即用 F1-Score 来衡量漏洞检测模型召回率和精确率的综合能力。F1-Score 如公式 5-3 所示。

$$Recall = \frac{TP}{TP + FN} \tag{5-1}$$

$$Precision = \frac{TP}{TP + FP} \tag{5-2}$$

$$F1 - Score = \frac{2 * Recall * Precision}{Recall + Precision} \tag{5-3}$$

如图 5-1 我们首先使用训练集 B 训练待评估模型，这里我们使用带噪声的训练集训练待评估模型，原因是我们在 4.1 节中已经证明了现有漏洞检测技术所使用的数据集中普遍存在噪声的现象，因此我们设置了可控的噪声比例，来模拟这种真实现状。但是我们使用测试集 A(干净数据集)进行评估并计算 F1-Score，这是因为，只有测试集是干净的才能得出真实的评估结果。

VOR (漏洞行覆盖率): 本课题研究源代码漏洞检测模型的鲁棒性的一个主要目的就是探究漏洞检测模型是否能够真正的学习到漏洞特征，一个真正根据漏洞的相关特征进行漏洞与非漏洞分类的模型，才是足够鲁棒的。我们认为通常与漏洞触发行有控制流或数据流相关的代码语句是识别漏洞与非漏洞的关键特征。我们使用近年来众多研究者比较关注的深度学习可解释性方法来辅助衡量漏洞检测模型的鲁棒性。因此，这一维度，我们通过深度学习可解释方法 (GNExplainer^[45] , SHAP^[46]) 对漏洞检测模型的预测结果进行解释，可解释方法，会反馈测试样本中影响模型决策的最重要的代码行，根据解释结果中是否包含漏洞行我们可以计算出 VOR。

$$VOR = \frac{\text{解释结果中包含漏洞行的测试样本数}}{\text{总测试样本数}} \tag{5-4}$$

如图 5-1 中第二条路径，与第一条路径不同之处在于，当我们使用测试集 A 输出测试结果之后，我们使用可解释方法对模型 A 的输出结果进行解释，并计算 VOR。接下来介绍可解释方法的技术细节。

在许多深度学习应用中，了解模型做出某种预测的原因与预测的准确性一样重要，如漏洞检测领域，不仅要知道一段代码是否有漏洞，知道其为什么被判定为漏洞对于软件开发人员修复漏洞也是极其重要的。然而，大型数据集的上得高精度往往是由复杂的模型实现的，即使是专家也很难解释这些模型，例如集成或深度学习模型，这在准确性和可解释性之间产生了紧张关系。作为回应，最近提

出了各种方法来帮助用户解释复杂模型的预测,但通常不清楚这些方法是如何关联的,以及何时一种方法优于另一种方法。为了解决这个问题,Lundberg 等人^[46]提出了一个解释预测的统一框架 SHAP (SHapley Additive exPlanations)。在进行局部解释时,SHAP 的核心是计算其中每个特征变量的 Shapley Value。SHapley:代表对每个样本中的每一个特征变量,都计算出它的 Shapley Value。Additive:代表对每一个样本而言,特征变量对应的 shapley value 是可加的。exPlanation:代表对单个样本的解释,即每个特征变量是如何影响模型的预测值。

当我们进行模型的 SHAP 事后解释时,我们需要明确标签。已知数据集(设有 M 个特征变量, n 个样本),原始模型 f ,以及原始模型 f 在数据集上的所有预测值。 g 是 SHAP 中用来解释 f 的模型。先用 f 对数据集进行预测,得到模型预测值的平均值 ϕ_0 。单个样本表示为 $x = (x_1, x_2 \dots x_M)$, $f(x)$ 为在原始模型下的预测值。 $g(x)$ 是事后解释模型的预测值,满足 $g(x) = \phi_0 + \sum_{i=1}^M \phi_i x_i = f(x)$ 。其中 ϕ_i 代表第 i 个特征变量的 Shapley Value,是 SHAP 中的核心要计算的值,需要满足唯一性。同时上述模型 g 需要满足局部保真性、确实性和连续性。

为了能够更好的抓取代码中得漏洞语义信息,许多漏洞检测模型采用了基于 GNN 的神经网络用来学习代码中的结构信息。因此,一般的可解释性方法通常不能适用于基于 GNN 的神经网络结构。为了解释图神经网络的预测结果,Ying 等人^[45]提出了 GNNExplainer。图形神经网络(GNNs)是一种强大的图形机器学习工具。GNN 通过沿输入图的边缘递归传递神经消息,将节点特征信息与图结构相结合。然而,结合图形结构和特征信息导致了复杂的模型,解释 GNN 做出的预测仍然没有解决。GNNExplainer,这是第一种通用的、模型无关的方法,用于在任何基于图的机器学习任务上为任何基于 GNN 的模型的预测提供可解释的解释。给定一个实例,GNNExplainer 确定了一个紧凑的子图结构和一小部分节点特征,这些特征在 GNN 的预测中起着至关重要的作用。此外,GNNExplainer 可以为整个实例类生成一致而简洁的解释。作者将 GNNExplainer 公式化为优化任务,最大化 GNN 预测和可能子图结构分布之间的相互信息。

因为 GNNExplainer 是模型无关的,所以任意基于 GNN 的机器学习模型都可以用 GNNExplainer 来解释,因此我们可以用其来解释 DeepWukong 的漏洞预测结果,获取影响样本判断为漏洞样本的关键代码行。

FR (标签翻转率):这一维度,我们侧重于漏洞检测模型在面对对抗攻击时的鲁棒性。一个足够鲁棒的漏洞检测模型,当面对相同语义的代码片段时,不应该给出不同的预测结果。否则,攻击者可以在不改变代码语义的情况下精心设计一些特殊输入案例,来绕过安全系统,最终可能会造成无法承担的损失。这里我们通过第四章提出的语义保护的代码变换技术,为测试集 A 生成相同数量大小

的对抗样本。若待评估模型对测试样本 a 极其 a 生成的对抗样本 b 给出了不同的预测结果，那么我们认为产生了标签翻转。在对测试集 A 和测试集 B 中全部样本进行预测之后，可以计算标签反转率 FR ，公式如 5-5。

$$FR = \frac{\text{标签翻转的样本数}}{\text{总测试样本数}} \quad (5-5)$$

如图 5-1 中第三条路径，使用训练集 B 训练后的待检测模型，分别在测试集 A 和 B 进行测试，测试集 A 和测试集 B 中的样本是一对一的，根据测试结果，计算出 FR 。

FD (F1 分数差)：最后，我们参考了 Chakraborty 等人^[9]的发现，训练完成的漏洞检测模型通常在与训练集同源的测试集上表现较好，当使用与训练集非同源的测试集进行测试评估时，性能会大幅度下降。通常我们认为，一个足够鲁棒的漏洞检测模型，应该对于同一种漏洞，无论数据源是否相同，都应该有着相同或者相近的评估结果，不应该有太大的性能差异。因此，最后一个维度，我们重点关注漏洞检测模型在非同源测试集上的性能差异，通过 $F1$ 分数差来进行评估，如公式 5-6。

$$FD = \frac{F1 - Score_A - F1 - Score_C}{F1 - Score_A} \quad (5-6)$$

如图 5-1 中的第四条路径，我们使用训练集 B 训练待评估模型，之后分别在测试集 A 和测试集 C 上计算 $F1 - Score_A$ 和 $F1 - Score_C$ ，最后得出 FD 得结果。

5.3 实验设置

本章中实验设置除了数据集选择、数据标注和噪声标签生成有些许不同之外，其余设置均与 3.3 描述相同。

5.3.1 数据集选择

本章中我们需要两个非同源的数据集来满足我们鲁棒性度量方法的需求，对于数据集 A 我们依然选择 $SARD$ 来进行实验，原因如下：

(1) $SARD$ 样本数量充足，且根据漏洞类型进行划分。

(2) $SARD$ 标签足够可靠，能够通过认为的设置可控比例的噪声来模拟现实世界数据的现状。

(3) SARD 提供了漏洞触发所在的行号, 方便我们计算 VOR (漏洞行覆盖率)

因为我们本章中的实验目的是为了验证鲁棒性度量方法及优化方法的有效性, 与漏洞类型的选择无关。因此我们选择 SARD 中的 CWE119 作为代表数据集进行参与实验。

对于数据集 C, 我们需要一个与 SARD 具有相同特点的漏洞数据集。之前的实验中我们选择了 FFMpeg+Qemu 作为真实数据集, 虽然它的标签足够可靠, 但是 FFMpeg+Qemu 仅提供了函数粒度的漏洞样本, 并没有提供标注到行的样本。因此我们选择了 D2A 作为数据集 C, 原因如下:

- (1) D2A 样本数量充足, 并且提供了漏洞样本的漏洞类型。
- (2) D2A 提供了漏洞的触发路径, 其中自然包括漏洞触发的行号。

因为我们仅需从 D2A 中抽样一部分数据作为测试集, 对于其标签质量, 我们可以通过人工审查的手段来过滤掉噪声样本。此外, D2A 中的 BufferOverRun 漏洞类型正好属于 CWE119, 虽然数据集不同源, 但是漏洞类型相同, 非常适合于本章的实验目的。

5.3.2 数据标注

SARD 数据标注的过程已经在 3.3.3 节中进行了详细描述。D2A 的样本标注, 可根据其提供的漏洞触发路径, 将其漏洞触发的最后一个代码行作为漏洞行, 包含该代码行的样本被标注为漏洞样本, 否则为非漏洞样本。

5.3.3 生成噪声标签

本章中, 我们需要模拟一个训练数据中存在噪声标签的事实情况, 因此, 我们对于干净的训练集 A 按照 3.3.4 节中的方式生成了 30% 的噪声标签得到训练集 B。从图 5-1 中可以看到, 本章中使用的数据除了训练集 B 是有噪声的, 其余都是干净的数据集, 尤其是用于测试的数据集, 我们保证其标签的可靠性, 以达到可靠的实验结果。

5.4 实验结果及结论

因为目前还未有针对漏洞检测模型的鲁棒性相关研究, 所以本章我们将结合第三章提出的基于集成学习与置信剪枝的数据集去噪方法, 以及第四章提出的基于语义保护代码变换的数据增强两种鲁棒性优化方法, 通过本章提出的鲁棒性评估方法来评估优化前后漏洞检测模型的鲁棒性。本章我们重点关注以下几个方面:

- (1) 经典的漏洞检测模型在我们评估方法下的表现。

- (2) 使用基于集成学习与置信剪枝去噪后对模型鲁棒性的影响。
- (3) 使用基于语义保护代码变换进行数据增强后对模型鲁棒性的影响。
- (4) 同时使用去噪和数据增强对模型鲁棒性的影响。

5.4.1 不同漏洞检测模型多维度鲁棒性评估

本次实验，我们将对 DeepWukong、SySeVr 和 VulDeePecker 三个经典的漏洞检测模型通过我们提出的鲁棒性评估方法来进行多维度的鲁棒性评估，分别从多个角度来比较三个漏洞检测模型的鲁棒性。

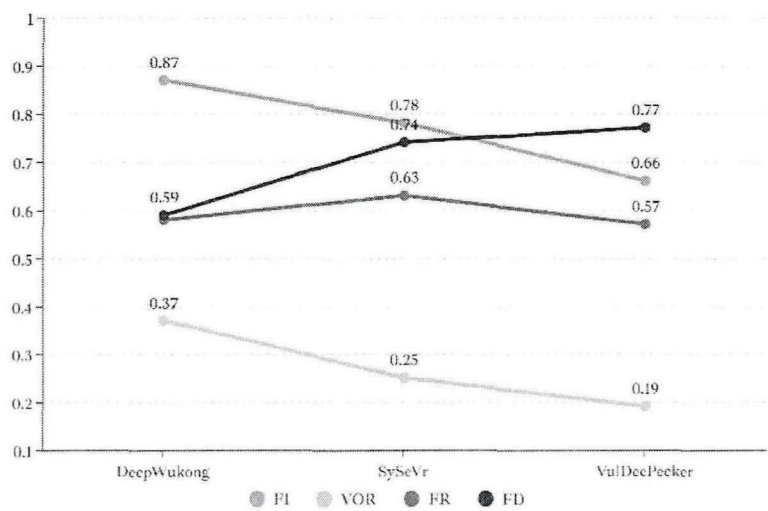


图 5-2 DeepWukong、SySeVr 和 VulDeePecker 的多维度鲁棒性评估

结果：如图 5-2 所示，我们用折线图表示了三个模型在四个不同比较维度上的鲁棒性表现。从评估模型漏洞检测能力的 F1 维度可以看出，在漏洞检测的能力表现上，DeepWukong 最好，VulDeePecker 最差，SySeVr 居中。从评估模型学习漏洞相关特征能力的 VOR 维度可以看到，在对模型预测结果进行解释之后，DeepWukong 的解释结果对于漏洞触发行的覆盖率最高，VulDeePecker 覆盖率最低，这说明 DeepWukong 学习漏洞特征的能力最强。从评估漏洞检测模型应对对抗攻击的能力 FR 来看，三个模型中标签反转率 SySeVr 最高，DeepWukong 和 VulDeePecker 相接近，这表明 SySeVr 最容易受到对抗样本攻击。最后，从评估漏洞检测模型的泛化能力 FD 来看，VulDeePecker 在非同源数据集上的表现最差，F1 下降了 77%，DeepWukong 的表现最好，但是也下降了 59%。综合四个维度来衡量模型鲁棒性，可以得出，DeepWukong 的鲁棒性相较于其他两个模型要更高一些，SySeVr 次之，VulDeePecker 最差。

分析：由于 DeepWukong 采用了基于 GNN 的模型结构，不仅考虑了代码的上下文信息，还考虑的代码与代码之间的语义结构关系，所以 DeepWukong 在

F1、VOR 和 FD 上表现都是最好的。而 SySeVr 和 VulDeePecker 采用的模型结构都是序列模型，因此只能从代码片段的上下文信息中提取关联，忽略了结构信息。因此鲁棒性要稍微弱一点，而 SySeVr 是在 VulDeePecker 基础上增加了控制流信息，考虑了更多的代码语义，因此要优于 VulDeePecker。而 DeepWukong 的 FR 较低原因是它的样本都是图形结构，经过代码变化，不仅改了文本内容，也改了图结构信息，相较于原始样本修改幅度较大，容易引起标签翻转。

结论：综合四个鲁棒性度量维度，DeepWukong 由于其基于图的模型结构有着最好的鲁棒性，SySeVr 由于其特征提取的优势鲁棒性高于 VulDeePecker。

5.4.2 去噪对模型鲁棒性的影响

本次实验，我们将通过本章提出的鲁棒性度量方法，来评估第三章提出的优化方法能否有效的在我们给出的四个维度上提升漏洞检测模型 DeepWukong 的鲁棒性。由于对多个模型进行评估，工作量太大，因此我们仅以最复杂的模型 DeepWukong 为代表进行接下来的实验。

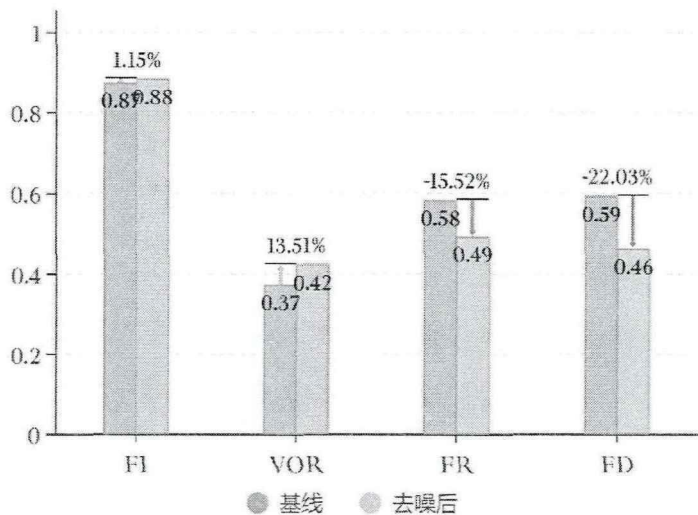


图 5-3 DeepWukong 在去噪前后的多维度鲁棒性评估

结果：如图 5-3 所示，在数据集去噪之后，DeepWukong 在 VOR 上从原来的 0.37 上升到了 0.42，这表明，去噪会使更多的测试样本在解释结果中包含了漏洞触发行。在标签翻转率 FR 上从原来的 0.58 降低到 0.49，这说明去噪后，受到对抗样本会攻击成功的样本减少了。在 FD 上可以看出，DeepWukong 去噪后泛化能力更强了，在非同源的测试集上性能下降的程度也有所减缓。

分析：在 3.4 节的实验中我们已经证明了训练集中存在噪声样本会对漏洞检测模型造成不可避免的影响，因此，本次实验中，当使用我们的去噪方法优化之后，模型在四个维度的鲁棒性评估指标都有所优化，这是因为，噪声对模型的负面影响通过去噪得到了减轻。

结论：在经过基于集成学习和置信剪枝的去噪方法优化后的数据集上重新训练模型能够有效提升模型的鲁棒性。

5.4.3 数据增强对模型鲁棒性的影响

本次实验，我们将通过本章提出的鲁棒性度量方法，来评估第四章提出的数据增强优化方法能否有效的在我们给出的四个维度上提升漏洞检测模型 DeepWukong 的鲁棒性。本次实验中我们直接选择全部变换规则作为数据增强策略，以达到最好的优化效果。

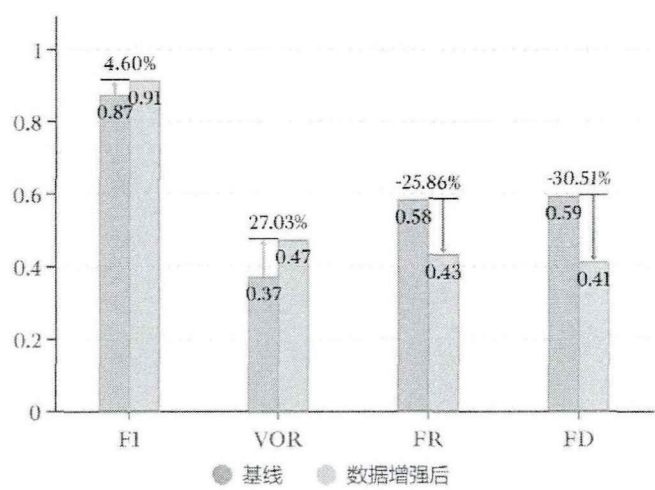


图 5-4 DeepWukong 在数据增强前后的多维度鲁棒性评估

结果：如图 5-4 在经过语义保护代码变换的数据增强之后，DeepWukong 在 VOR 上从原来的 0.37 上升到了 0.47，在标签翻转率 FR 上从原来的 0.58 降低到 0.43，在 FD 上从原来的 0.59 降低到了 0.41，且在 F1 上也有了 4.6%的提升，数据增强之后鲁棒性的提升比去噪后还要高一些。

分析：我们通过数据增强来扩充原本模式单一且数量有限的训练样本集合，得到一个代码模式多样且样本数量充足的数据集，重新训练之后的模型能够极大的提升其泛化能力，且模型对于漏洞特征的学习提取也更加精准了，因此数据增强带来的优化比去噪还要有效。

结论：在经过语义保护代码变换的数据增强方法优化后的数据集上重新训练模型能够有效提升模型的鲁棒性。

5.4.4 数据增强结合去噪对模型鲁棒性的影响

我们前面已经证明了单独使用去噪方法和数据增强方法优化模型的鲁棒性是有效的，那么本次实验我们将综合使用两种方法共同优化，来验证其对模型鲁棒性的影响。

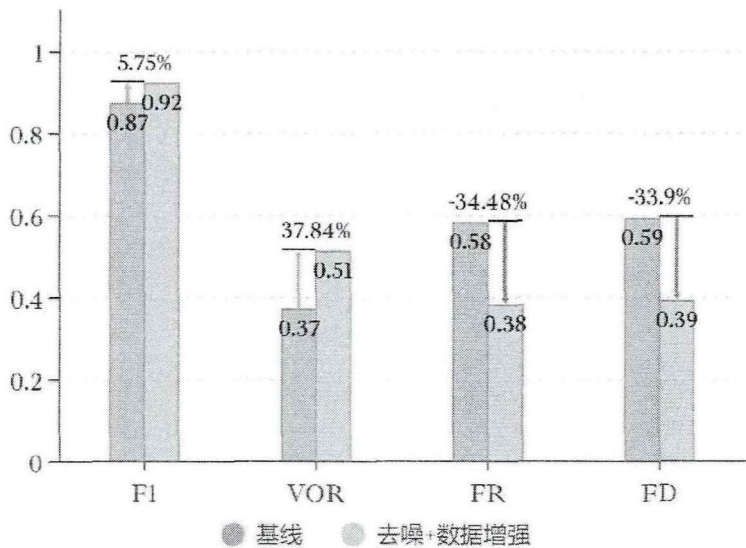


图 5-5 DeepWukong 在数据增强结合去噪前的多维度鲁棒性评估

结果：如图 5-5 所示，在同时使用两种优化方法之后，DeepWukong 在各个维度都优于单独使用一种方法，这同我们设想相同。尤其是在 FR 标签翻转率的评估表现上，直接从 0.58 下降到 0.38，将攻击成功的对抗样本降低到 38%。此外，在漏洞行覆盖率 VOR 上的表现也很优秀，能够有 51% 的解释结果包含漏洞触发行，这说明影响模型决策的重要因素逐渐变成了与漏洞更相关的代码行。

分析：将去噪方法和数据增强相结合对于模型鲁棒性的优化是可以很直观的理解的，因为我们在 0 中就已经抛出了常用数据集中存在噪声的普遍现象，以及漏洞样本形式单一且缺乏会导致模型鲁棒性低的问题，因此，我们在第三章和第四章分别提出了两种方法来解决这两个问题，这两种方法在单独使用时是有效的，且它们互相并不冲突，因此同时使用时也能够同时发挥各自的作用，进一步提升模型的鲁棒性。

结论：在同时使用语义保护代码变换的数据增强方法和基于集成学习和置信剪枝的去噪方法优化后的数据集上重新训练模型能够进一步提升模型的鲁棒性。

5.5 本章小结

在本章节中,本课题针对现阶段基于学习的漏洞检测领域内研究者们致力于提升漏洞检测模型的精度而忽略其鲁棒性研究的问题,提出了一个多维度鲁棒性度量方法,通过四个维度来综合评估了漏洞检测模型 DeepWukong、SySeVr 和 VulDeePecker 的鲁棒性,证实了 DeepWukong 鲁棒性要强与其他两个模型。其次,我们使用我们的评估方法对前两章提出的优化方法进行了有效性评估,证明了基于集成学习和置信剪枝的去噪方法和基于语义保护代码变换的数据增强方法都能够有效的提升漏洞检测模型的鲁棒性。最后,我们还证明了去噪方法和数据增强对模型鲁棒性改善是不冲突的,可以共同使用,我们提出的鲁棒性度量方法可能不够全面,但足以为后续的研究工作提供一定的参考价值。

第六章 总结与展望

6.1 论文总结

随着开源软件供应链的日益复杂,开源软件中存在的源代码安全漏洞也会造成更加深远的恶劣影响,因此研究一个能够精准检测开源软件中漏洞的工具变得越来越迫切。由于现阶段 AI 技术在 CV 和 NLP 领域的兴起,引发了众多研究者将深度学习技术应用到源代码漏洞检测中去。然而大家都在趋之若鹜的朝着提升漏洞检测模型的精度触发,忽略了最重要的问题,模型鲁棒性。在软件安全领域中一个不够鲁棒的模型是无法真实应用的,一旦在生产环境中出现问题,造成的后果将不堪设想。

本课题参考了众多漏洞检测相关研究工作,指出了基于深度学习的漏洞检测任务中,研究者们常用的漏洞数据集中存在一些严重的问题会影响下游模型的鲁棒性。第一个问题是由于不完善的收集过程及使用方式,这些数据集中包含大量噪声标签,这将影响模型在实际应用时的决策。第二个问题是常用的漏洞数据集中样本模式单一且不易收集,因而数据量不足,这将导致训练出的模型泛化性不足。这两个问题都会影响模型的鲁棒性。

针对数据集包含噪声的问题,我们证实了数据集中普遍存在噪声的现状,并探究了噪声的来源,且证实了数据集中存在噪声确实会导致模型决策错误。因为数据集收集过程中产生噪声标签几乎是不可避免的,因此提出一个收集干净数据的方法几乎是不可完成的任务,所以我们决定通过去噪来降低噪声的影响。我们提出了基于集成学习和置信剪枝的数据集去噪方法,并证明了通过去噪能够提升模型检测漏洞的性能。

针对数据集样本单一并且数据量少的问题,我们借鉴了其他机器学习任务中解决此类问题的方法——数据增强。因为源代码自带的语法及语义属性,我们无法像其他机器学习任务中简单的进行数据增强,我们需要在保证代码语义不变的情况下生成与原来不同的样本。我们提出了基于语义保护的代码变换技术来进行数据增强,并提出了两种数据增强的策略。我们证明了数据增强在提升漏洞模型检测能力上的有效性,并且证明了数据量增加对提升模型性能也是有帮助的。

最后,鉴于领域内没有系统得衡量漏洞检测模型鲁棒性得方法,我们借鉴了其他领域研究模型鲁棒性得方法,提出了多维度的漏洞检测模型鲁棒性度量方法。我们评估了经典漏洞检测模型的鲁棒性,并证明了我们提出的两种优化方法的有效性,以及度量方法的合理性。

6.2 未来工作展望

本课题针对基于学习的源代码漏洞检测领域中对于鲁棒性研究的空缺，提出了提升鲁棒性的两个优化方法，以及度量鲁棒性的一个方法。

首先我们针对漏洞数据集中普遍存在噪声这个问题上提出了一个基于集成学习和置信剪枝的去噪方法，这个方法虽然有效，但这是作为一个弥补方案来进行的。最好的方法是在漏洞收集和使用的時候，采用更加精准的方法，让我们收集数据的过程中就将噪声数据过滤掉。因此，在未来的工作中，我们会把工作中心放到优化漏洞数据收集方法上，争取为漏洞检测领域产出一个优质的数据集作为未来相关工作的 BaseLine。

其次我们针对漏洞数据收集困难且样本模式单一的缺点，提出了基于语义保护代码变换的数据增强方式来扩充样本多样性以及数据量。但是根据我们实验得出的结论，并非所有的代码变换规则都能够对模型学习有帮助，也并非数据量越大，模型学习的漏洞特征越准确。因此，这种随机的代码变换并不能作为一种高效的数据增强方式，我们希望能够有针对性的对代码进行修改，以达到最佳效果。未来的工作中，我们会以可解释性方法的反馈结果作为辅助，在训练过程中有针对性的对训练样本进行代码变换，帮助模型一步一步学习到漏洞的真实特征。此外，除了数据增强，我们也将考虑使用 BUG 注入技术在真实代码中注入一些真实漏洞以获得更丰富的漏洞样本。

最后我们提出了多维度的漏洞检测模型鲁棒性度量方法，虽然我们参考了其他领域的鲁棒性研究工作，并提出了四个鲁棒性度量的维度。这四个维度都是漏洞检测领域比较重要且需要关注的角度，但是覆盖是否全面还需后续研究使用中进一步完善。近年来基于深度学习的漏洞检测模型层出不穷，本课题仅对几个具有代表性的模型进行了评估对比，未来的工作中还会对一些新兴的漏洞检测模型进行评估比较。

参考文献

- [1] Johnson B, Song Y, Murphy-Hill E, et al. Why don't software developers use static analysis tools to find bugs?[C]//2013 35th International Conference on Software Engineering (ICSE). IEEE, 2013: 672-681.
- [2] Smith J, Johnson B, Murphy-Hill E, et al. Questions developers ask while diagnosing potential security vulnerabilities with static analysis[C]//Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. 2015: 248-259.
- [3] Ayewah N, Pugh W, Morgenthaler J D, et al. Evaluating static analysis defect warnings on production software[C]//Proceedings of the 7th ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools and engineering. 2007: 1-8.
- [4] Newsome J, Song D X. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software[C]//NDSS. 2005, 5: 3-4.
- [5] Liu B, Shi L, Cai Z, et al. Software vulnerability discovery techniques: A survey[C]//2012 fourth international conference on multimedia information networking and security. IEEE, 2012: 152-156.
- [6] Li M, Li C, Li S, et al. ACGVD: vulnerability detection based on comprehensive graph via graph neural network with attention[C]//Information and Communications Security: 23rd International Conference, ICICS 2021, Chongqing, China, November 19-21, 2021, Proceedings, Part I 23. Springer International Publishing, 2021: 243-259.
- [7] Nguyen V A, Nguyen D Q, Nguyen V, et al. ReGVD: Revisiting graph neural networks for vulnerability detection[C]//Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. 2022: 178-182.
- [8] Wang H, Ye G, Tang Z, et al. Combining graph-based learning with automated data collection for code vulnerability detection[J]. IEEE Transactions on Information Forensics and Security, 2020, 16: 1943-1958.
- [9] Chakraborty S, Krishna R, Ding Y, et al. Deep learning based vulnerability detection: Are we there yet[J]. IEEE Transactions on Software Engineering,

- 2021.
- [10] Li Z, Zou D, Xu S, et al. Vuldeepecker: A deep learning-based system for vulnerability detection[J]. arXiv preprint arXiv:1801.01681, 2018.
 - [11] Hu Y. A framework for using deep learning to detect software vulnerabilities[J]. 2019.
 - [12] Zou D, Wang S, Xu S, et al. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection[J]. IEEE Transactions on Dependable and Secure Computing, 2019, 18(5): 2224-2236.
 - [13] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//2018 17th IEEE international conference on machine learning and applications (ICMLA). IEEE, 2018: 757-762.
 - [14] Zhou Y, Liu S, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in neural information processing systems, 2019, 32.
 - [15] Cheng X, Wang H, Hua J, et al. Static detection of control-flow-related vulnerabilities using graph embedding[C]//2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS). IEEE, 2019: 41-50.
 - [16] Cheng X, Wang H, Hua J, et al. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2021, 30(3): 1-33.
 - [17] Springenberg J T, Dosovitskiy A, Brox T, et al. Striving for simplicity: The all convolutional net[J]. arXiv preprint arXiv:1412.6806, 2014.
 - [18] Guo W, Mu D, Xu J, et al. Lemna: Explaining deep learning based security applications[C]//proceedings of the 2018 ACM SIGSAC conference on computer and communications security. 2018: 364-379.
 - [19] Li Z, Tang J, Zou D, et al. Towards Making Deep Learning-based Vulnerability Detectors Robust[J]. arXiv preprint arXiv:2108.00669, 2021.
 - [20] Lecuyer M, Atlidakis V, Geambasu R, et al. Certified robustness to adversarial examples with differential privacy[C]//2019 IEEE Symposium on Security and Privacy (SP). IEEE, 2019: 656-672.
 - [21] Salman H, Sun M, Yang G, et al. Black-box smoothing: A provable defense for pretrained classifiers[J]. arXiv preprint arXiv:2003.01908, 2020, 2(2).

- [22] Jia J, Cao X, Wang B, et al. Certified robustness for top-k predictions against adversarial perturbations via randomized smoothing[J]. arXiv preprint arXiv:1912.09899, 2019.
- [23] Wang B, Cao X, Gong N Z. On certifying robustness against backdoor attacks via randomized smoothing[J]. arXiv preprint arXiv:2002.11750, 2020.
- [24] Weber M, Xu X, Karlaš B, et al. Rab: Provable robustness against backdoor attacks[J]. arXiv preprint arXiv:2003.08904, 2020.
- [25] Chakraborty J, Majumder S, Menzies T. Bias in machine learning software: Why? how? what to do?[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021: 429-440.
- [26] Northcutt C, Jiang L, Chuang I. Confident learning: Estimating uncertainty in dataset labels[J]. Journal of Artificial Intelligence Research, 2021, 70: 1373-1411.
- [27] Xu J, Li Y, Deng R H. Differential training: A generic framework to reduce label noises for android malware detection[J]. 2021.
- [28] Suneja S, Zheng Y, Zhuang Y, et al. Towards reliable ai for source code understanding[C]//Proceedings of the ACM Symposium on Cloud Computing. 2021: 403-411.
- [29] Rong X. word2vec parameter learning explained[J]. arXiv preprint arXiv:1411.2738, 2014.
- [30] Zhang C, Bengio S, Hardt M, et al. Understanding deep learning (still) requires rethinking generalization[J]. Communications of the ACM, 2021, 64(3): 107-115.
- [31] Amid E, Warmuth M K, Srinivasan S. Two-temperature logistic regression based on the tsallis divergence[C]//The 22nd International Conference on Artificial Intelligence and Statistics. PMLR, 2019: 2388-2396.
- [32] Amid E, Warmuth M K K, Anil R, et al. Robust bi-tempered logistic loss based on bregman divergences[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [33] Li Y, Wang S, Nguyen T N. Vulnerability detection with fine-grained interpretations[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of

- Software Engineering. 2021: 292-303.
- [34] Wang H, Ye G, Tang Z, et al. Combining graph-based learning with automated data collection for code vulnerability detection[J]. IEEE Transactions on Information Forensics and Security, 2020, 16: 1943-1958.
- [35] Zheng Y, Pujar S, Lewis B, et al. D2a: A dataset built for ai-based vulnerability detection methods using differential analysis[C]//2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2021: 111-120.
- [36] Fan J, Li Y, Wang S, et al. AC/C++ code vulnerability dataset with code changes and CVE summaries[C]//Proceedings of the 17th International Conference on Mining Software Repositories. 2020: 508-512.
- [37] Gkortzis A, Mitropoulos D, Spinellis D. VulinOSS: a dataset of security vulnerabilities in open-source systems[C]//Proceedings of the 15th International conference on mining software repositories. 2018: 18-21.
- [38] Nikitopoulos G, Dritsa K, Louridas P, et al. CrossVul: a cross-language vulnerability dataset with commit data[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021: 1565-1569.
- [39] Wang X, Sun K, Batcheller A, et al. Detecting" 0-day" vulnerability: An empirical study of secret security patch in OSS[C]//2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). IEEE, 2019: 485-492.
- [40] Sabetta A, Bezzi M. A practical approach to the automatic classification of security-relevant commits[C]//2018 IEEE International conference on software maintenance and evolution (ICSME). IEEE, 2018: 579-582.
- [41] Zhou Y, Sharma A. Automated identification of security issues from commit messages and bug reports[C]//Proceedings of the 2017 11th joint meeting on foundations of software engineering. 2017: 914-919.
- [42] Perl H, Dechand S, Smith M, et al. Vccfinder: Finding potential vulnerabilities in open-source projects to assist code audits[C]//Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security. 2015: 426-437.
- [43] Nikitopoulos G, Dritsa K, Louridas P, et al. CrossVul: a cross-language vulnerability dataset with commit data[C]//Proceedings of the 29th ACM

- Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021: 1565-1569.
- [44] Sabetta A, Bezzi M. A practical approach to the automatic classification of security-relevant commits[C]//2018 IEEE International conference on software maintenance and evolution (ICSME). IEEE, 2018: 579-582.
- [45] Ying Z, Bourgeois D, You J, et al. Gnnexplainer: Generating explanations for graph neural networks[J]. Advances in neural information processing systems, 2019, 32.
- [46] Lundberg S M, Lee S I. A unified approach to interpreting model predictions[J]. Advances in neural information processing systems, 2017, 30.
- [47] Cao S, Sun X, Bo L, et al. MVD: memory-related vulnerability detection based on flow-sensitive graph neural networks[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 1456-1468..
- [48] Hin D, Kan A, Chen H, et al. LineVD: statement-level vulnerability detection using graph neural networks[C]//Proceedings of the 19th International Conference on Mining Software Repositories. 2022: 596-607.
- [49] Zhou Y, Sharma A. Automated identification of security issues from commit messages and bug reports[C]//Proceedings of the 2017 11th joint meeting on foundations of software engineering. 2017: 914-919.
- [50] Wang Y, Ma X, Chen Z, et al. Symmetric cross entropy for robust learning with noisy labels[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. 2019: 322-330.
- [51] Ponta S E, Plate H, Sabetta A, et al. A manually-curated dataset of fixes to vulnerabilities of open-source software[C]//2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR). IEEE, 2019: 383-387.
- [52] Lu S, Guo D, Ren S, et al. Codexglue: A machine learning benchmark dataset for code understanding and generation[J]. arXiv preprint arXiv:2102.04664, 2021.